Uncertainty-Aware Autonomous Driving: HD-Map Localization, Behaviour Trees and Adaptive MPC

M.Tech Project ReportPython 3 / NumPy / SciPy code base; CARLA and ROS 2 integration pending

November 4, 2026

Abstract

This report documents a Python implementation of an autonomy stack: NDT + particle-filter localization on an HD map, a behaviour tree (BT) for overtaking, roundabout entry and yielding, a Frenet space-time lattice planner with QP trajectory optimisation, and pure-pursuit / Stanley / adaptive-MPC tracking. Every number reported here was produced by the scripts in this repository on *synthetic* worlds; the CARLA, ROS 2, public-dataset and embedded-hardware experiments are written but **not yet run** and are listed separately. The system is a research prototype and is **not** validated for production use.

Keywords: HD-map localization, NDT, particle filter, behaviour tree, Frenet lattice, MPC, fault injection.

1 Objectives

1. Localize against a pre-built HD map using NDT scan matching fused in a particle filter.
2. Decide overtaking, roundabout entry and yielding with a behaviour tree that can be checked automatically.
3. Plan collision-free, smooth trajectories (lattice + QP) and track them under actuator and localization faults.
4. Quantify behaviour under injected faults and report failures honestly; keep all experiments reproducible.

2 System Architecture

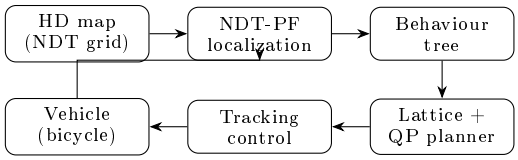


Figure 1: Data flow. The BT outputs a target speed and lane; the planner turns them into a trajectory; the controller tracks it using the localized pose.

3 Methodology and Equations

3.1 Localization

The HD map points are binned into cells with mean μ_c and covariance Σ_c (regularised by $0.15I$). For pose $x = (p_x, p_y, \theta)$ and scan points y_i, the NDT score is

$$S(x) = \sum_i \exp\!\big(-\tfrac{1}{2}(T(x,y_i)-\mu_{c(i)})^\top \Sigma_{c(i)}^{-1}(T(x,y_i)-\mu_{c(i)})\big). \quad (1)$$

Each particle is weighted $w_j \propto w_j\,(S(x_j)+\epsilon)^\beta$ with $\beta = 0.5$. *NDT-PF* adds (i) a damped-Newton NDT refinement of the weighted mean; if it scores > 0.02 per point higher, 20% of the lowest-weight particles are reseeded around it, and (ii) process-noise inflation $\times 3$ after 5 consecutive steps with normalised score < 0.25.

3.2 Behaviour tree

Tick semantics: *Selector* returns the first non-FAILURE child and resets lower-priority children; *Sequence* with memory resumes the running child. Root order: roundabout $\succ$ overtake $\succ$ cruise. Overtaking starts only if the predicted pass (kinematic, duration t_p, ego distance d_e) satisfies

$$g_{onc} - d_e - |v_{onc}|\,k\,t_p > m, \quad (k,m) = (1.3,\ 40\ \text{m}) \quad (2)$$

and aborts with hysteresis $(k,m) = (1.0, 15\ \text{m})$ only if merging back is safe. Roundabout yielding compares the predicted arrival time t_a at the conflict point with circulating-vehicle passage times t_c: conflict if $|t_c - t_a| < w$, $w = 3$ s (4.5 s near the point of no return).

3.3 Planning

Lateral quintic and longitudinal quartic/quintic polynomials are sampled over a lattice of terminal lateral offset l_f, terminal speed v_f and horizon $T \in \{3,4,5\}$ s, plus follow-lead terminals. Candidates are rejected on acceleration, lateral acceleration, road limits and space-time collision, then ranked by

$$J = w_j \int (\dddot{l}^{\,2} + \dddot{s}^{\,2})dt + w_T T + w_v(\dot{s}_f - v_{des})^2 + w_l\, d(l_f, \text{lane}). \quad (3)$$

Table 1: Modules (all code is listed in the appendix).

File	Content
`localization.py`	map, NDT grid, Newton alignment, NDT-PF
`behavior_tree.py`	BT library, 1-D driving world, driving tree
`verify_bt.py`	structural checks, runtime monitors, grid verification
`planning.py`	lattice planner, QP optimiser, limit checks
`control.py`	reference road, bicycle model, 3 controllers
`closed_loop.py`	planner→controller→vehicle evaluation
`fault_injection.py`	localization and decision fault campaigns
`datasets.py`	KITTI loader, ATE/RPE (synthetic-tested only)
`carla_bt_runner.py`	CARLA adapter (not yet run)
`deploy/`	benchmark, ROS 2 node, Docker

The best candidate is re-optimised as a bounded least-squares problem on the sample vector $x \in \{s(t), l(t)\}$:

$$\min_x \; w_j \|D_3 x\|^2 + w_d \|x - x_{lat}\|^2 \quad \text{s.t. } x_{lo} \le x \le x_{hi} \quad (4)$$

with corridor bounds from obstacle sides; the result is used only if collision-free and within dynamic limits.

3.4 Control

Error model about the reference with steering gain k: $\dot{e}_y = v e_\theta$, $\dot{e}_\theta = \frac{kv}{L} u$, $u = \delta - \delta_{ff}$, $\delta_{ff} = \arctan(L\kappa)/\hat{k}$. The MPC solves

$$\min_u \sum_{j=1}^{N} \left(q_y e_{y,j}^2 + q_\theta e_{\theta,j}^2\right) + r\|u\|^2 + r_d \|\Delta u\|^2, \; |\delta| \le 0.5 \quad (5)$$

($N = 15$, $\Delta t = 0.1$ s). The gain is adapted by NLMS from gyro yaw rate r_m with regressor $\phi = \frac{v}{L} \tan \delta_{filt}$:

$$\hat{k} \leftarrow \hat{k} + \mu \, \phi \, (r_m - \hat{k}\phi)/(\epsilon + \phi^2). \quad (6)$$

4 Implementation

5 Experiments Completed

All runs: single CPU x86-64, Python 3.12.3, NumPy 2.4.4; raw logs in Appendix C. Worlds are synthetic (Sec. 11).

5.1 E1: Localization (5 seeds, 412 m route)

Sensor: 80-point scans, $\sigma = 0.1$ m, 10% outliers; gyro bias 0.01 rad/s.

Table 2: E1 position RMSE [m] / converged (final error < 1 m).

Method	Nominal init	Hard init (5 m, 20°)
Dead reckoning	35.01 / 0%	53.78 / 0%
NDT only	0.11 / 100%	0.49 / 100%
PF (NDT likelihood)	2.89 / 0%	3.04 / 0%
NDT-PF	0.30 / 100%	0.50 / 100%

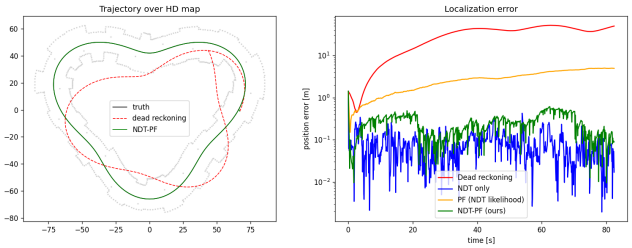


Figure 2: E1 trajectory and error (first seed).

5.2 E2: Behaviour tree, 300 random episodes

Random lead speed/position, oncoming car and 2–4 circulating vehicles. Result: 100% completion, 0% collisions, overtakes in 72% of episodes, mean route time 57.5 s, minimum gap median 9.2 m (5th percentile 6.4 m), waiting at the yield line in 35% of episodes.

5.3 E3: Behaviour-tree verification

The tree has 11 leaves, depth 4, and a total root (last child never fails). Monitors: no collision, TTC ≥ 3 s in the oncoming lane, speed ≤ 8 m/s in the roundabout, no lane change near it, one active leaf per tick. Every grid point was simulated: 3 575 overtaking and 1 856 roundabout scenarios gave **0 violations**. This is bounded verification over the grid and the 1-D model, not a proof for real traffic. During development the same procedure exposed defects (Table 3); the violation count on the overtaking grid went $148 \to 26 \to 22 \to 15 \to 135 \to 0$ over successive fixes.

5.4 E4: Fault injection

The score-based divergence detector (score < 0.25 for 5 steps): recall 0.70 / false-alarm 0.11 in the 20 s blackout, recall 1.00 / false-alarm 0.19 in the severe combo, and false-alarm 0.94 at 70% outliers (the fixed threshold is too conservative); elsewhere no step exceeded 2 m error so recall is undefined.

The supervisor flag is derived from the injected fault (idealised detector); a real detector is not implemented. Completion was 100% in all cases.

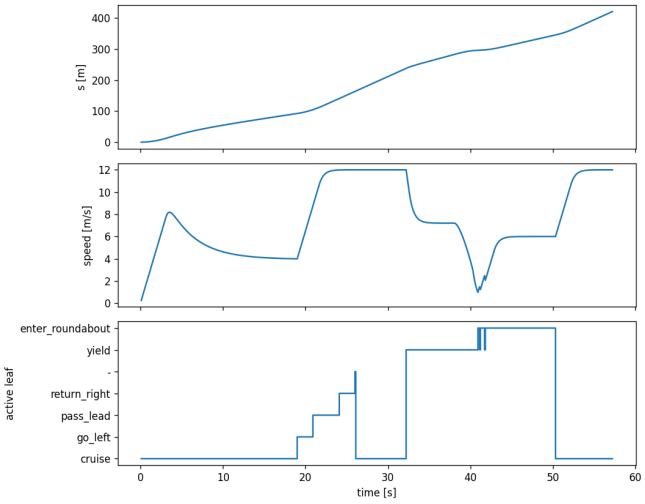


Figure 3: E2 one episode: position, speed and active leaf.

Table 3: Defects found by verification/random testing.

Defect	Fix
oncoming car counted as "passed" when beside ego	clearance of two car lengths + 5 m
lead gap undefined once ego in left lane ⇒ chatter	lane-independent gap, hysteresis
optimistic pass time ⇒ unsafe abort into lead	kinematic prediction, merge-safety test
roundabout branch preempted a pass	pass must end before approach zone
no car-following in roundabout actions	speed cap applied to all actions
yield overshoot into conflict zone	aim 2 m short, wider window
scripted lead rear-ended stopped ego (model flaw)	IDM follower

5.5 E5: Closed loop (planner → controller)

Scenario: slow lead (5 m/s, $s = 45$) and a parked car in the ego lane ($s = 180$); target 12 m/s; 330 m. Conditions: steering gain 0.7 with 0.1 s latency, localization noise ($\sigma_{xy} = 0.3$ m, $\sigma_\theta = 0.01$ rad), and both. Observations: (i) the planner overtakes the lead and passes the parked car in all conditions; the QP result was rejected (fallback to the lattice trajectory) in 3 of 57 replans in nominal runs; (ii) fixed-model MPC gave the lowest error in three of four conditions; (iii) the gain estimator tracked the true gain without latency (0.98–1.01 nominal) but **degraded** the system with latency: the estimate fell to 0.51 (true 0.7) because the regressor ignores the 0.1 s delay, producing oscillation. This is an unresolved failure of the adaptive scheme as implemented; no benefit of adaptation was demonstrated in this study.

Table 4: E4a localization faults: RMSE [m] / converged, 5 seeds. Max error 1.41 m in the first rows is the initial offset.

Fault	NDT only	NDT-PF
baseline	0.11 / 100%	0.30 / 100%
blackout 5 s	0.16 / 100%	0.35 / 100%
outliers 40%	0.14 / 100%	0.32 / 100%
gyro bias 0.03	0.12 / 100%	0.21 / 100%
noise 0.3 m	0.16 / 100%	0.29 / 100%
map change 33%	0.14 / 100%	0.41 / 100%
blackout 20 s	29.64 / 20%	**2.77 / 100%**
outliers 70%	0.23 / 100%	0.38 / 100%
map change 67%	0.17 / 100%	0.35 / 100%
severe combo	61.09 / 0%	54.47 / 0%

Table 5: E4b perception faults into the BT, 200 episodes each: collision rate without / with degraded-mode supervisor.

Fault	No supervisor	Supervisor
none	0.0%	0.0%
latency 0.5 s	0.0%	0.0%
latency 1.0 s	0.0%	0.0%
oncoming dropout 3 s	**86.5%**	0.0%
oncoming speed ×0.6	0.0%	0.0%
circulating timing $\sigma = 1.5$ s	2.0%	2.0%

5.6 E6: Latency (host CPU, not embedded)

Latency figures predate the final planner cost retuning; re-run `deploy/embedded_bench.py` before quoting them.

5.7 E7: Software tests

`tests/test_all.py` (6 tests: BT safety, oncoming rule, NDT recovery, planner feasibility, emergency stop, controller convergence) and `tests/test_datasets.py` pass.

6 Experiment Log (extend here)

7 Experiments Planned This Week (not yet run)

- **E8 CARLA:** run `carla_bt_runner.py` (written, never executed) for 20 seeds; record collisions, completion, rule violations.
- **E9 KITTI:** run NDT-PF on odometry sequence 00 with `datasets.py`; report ATE/RPE. The loader has only been tested on synthetic KITTI-format files.
- **E10 Embedded:** run the latency benchmark on the target board and compare with the 10 Hz budget (100 ms).

Table 6: E5 RMS tracking error e_y [m] / max $|e_y|$ [m] / minimum clearance [m]; collisions were 0 in all 16 runs. †: did not finish within 60 s.

Condition	Pure pursuit	Stanley	MPC (fixed)	Adaptive MPC
nominal	0.112/0.51/1.44	0.070/0.26/1.29	0.060/0.24/1.18	0.060/0.24/1.17
gain 0.7 + latency 0.1 s	0.160/0.61/1.39	0.117/0.51/1.31	0.090/0.36/1.24	**0.897/3.08/0.81**
loc. noise 0.3 m	0.193/0.64/1.04	0.155/0.62/0.79	0.145/0.62/1.35	0.145/0.62/1.35
all faults	0.254/0.91/1.06	0.205/0.67/1.49	0.217/0.59/1.60	**0.618/4.57/0.40**†

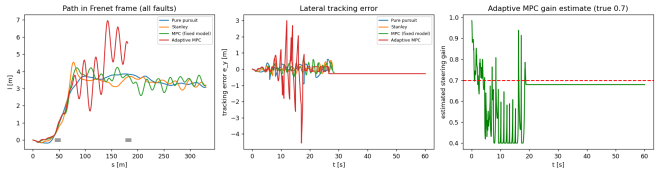


Figure 4: E5 (all faults): path, tracking error, adaptive gain estimate.

Table 7: E6 per-call latency [ms], 60–500 calls.

Module	p50	p95
NDT-PF cycle, 300 particles	5.61	6.22
PF update, 1000 particles	11.95	13.86
NDT alignment (6 iters)	1.92	2.27
Lattice planner	20.48	30.12
QP optimisation	0.88	1.00
MPC step	0.18	0.24
BT tick	0.03	0.04

- **E11 Adaptive MPC fix:** include actuator latency in the estimator regressor, freeze adaptation when $|\phi|$ or delay is uncertain, and repeat E5.

8 Future Experiments, Improvements and Observations

- NDT-only beat NDT-PF in every non-blackout case; the particle filter's value appeared only in long sensor dropouts. Test other ambiguity cases (symmetric streets, kidnapping) before claiming more.
- Adaptive divergence threshold for the localization monitor (false alarms 94% at 70% outliers); replace the idealised BT supervisor with a real perception-health monitor.
- Circulating-traffic timing error caused 2% collisions: use probabilistic gap acceptance.
- Planner replans cause reference discontinuities; add warm-start continuity costs. QP rejected in 5% of replans.
- Replace the 1-D BT world with CARLA scenarios, add pedestrians and multi-lane roundabouts, and use real perception.
- Report confidence intervals with more seeds (5 seeds in E1, E4a).

9 Safety Argument (current evidence and gaps)

No hazard analysis (ISO 26262 / SOTIF), real-world data, or hardware testing exists. The prototype must not be deployed on a road vehicle.

10 Real-World Applications

Localization against HD maps for urban driving and shuttles; rule-based, inspectable decision making for overtaking, roundabouts and yielding; trajectory planning and tracking for ADAS/L4 research; fault-injection methods for validation of autonomy components.

11 Limitations

Synthetic 2-D map with perfect data association assumptions, 1-D driving world with scripted traffic, kinematic vehicle with perfect obstacle perception (planner), 5 seeds for localization, single-CPU host timings. Results do not transfer to real roads without further work.

References

[1] P. Biber, W. Straßer, "The normal distributions transform: A new approach to laser scan matching," IROS, 2003.

[2] S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics*, MIT Press, 2005.

[3] M. Colledanchise, P. Ögren, *Behavior Trees in Robotics and AI*, CRC Press, 2018.

[4] M. Werling et al., "Optimal trajectory generation for dynamic street scenarios in a Frenet frame," ICRA, 2010.

[5] G. Hoffmann et al., "Autonomous automobile trajectory tracking for off-road driving," ACC, 2007.

Table 8: Experiment register. Add one `\explog` line per new experiment.

ID	Status	Script	Output	Summary
E1	done	`run_localization.py`	`results/localization.log`	NDT-only best nominal; PF alone does not converge
E2	done	`behavior_tree.py`	`results/bt_batch.log`	300 episodes, 0 collisions
E3	done	`verify_bt.py`	`results/bt_verification.log`	5 431 grid scenarios, 0 violations
E4	done	`fault_injection.py`	`results/fault_injection.log`	blackout 20 s: NDT-PF recovers, NDT diverges
E5	done	`closed_loop.py`	`results/closed_loop.log`	adaptive MPC fails with latency
E6	done	`deploy/embedded_bench.py`	`results/latency.log`	host latency only
E7	done	`tests/`	`results/test_all.log`	7 tests pass
E8	planned	`carla_bt_runner.py`	-	BT in CARLA Town04, 20 seeds
E9	planned	`datasets.py`	-	NDT-PF ATE on KITTI odometry seq. 00
E10	planned	`deploy/embedded_bench.py`	-	latency on target hardware
E11	planned	`control.py`	-	model latency in gain estimator; re-run E5

Table 9: Claims, evidence and status.

Claim	Evidence	Status
BT never collides in modelled scenarios	E2, E3 (grid, 1-D)	partial
BT safe under perception faults	E4b	dropout needs detector
Localization recovers from dropouts	E4a (20 s)	partial; combo fails
Tracking remains safe under faults	E5	adaptive MPC fails
Real-time on target	E6 host only	not shown
Works in real traffic	none	**open**

[6] J. Snider, "Automatic steering methods for autonomous automobile path tracking," CMU-RI-TR-09-08, 2009.

[7] A. Dosovitskiy et al., "CARLA: An open urban driving simulator," CoRL, 2017.

[8] A. Geiger, P. Lenz, R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," CVPR, 2012.

A Folder Structure and Reproduction

```
sdc_project/
  main.tex  Makefile  requirements.txt  run_all.py
  localization.py  run_localization.py
  behavior_tree.py  verify_bt.py
  planning.py  control.py  closed_loop.py
  fault_injection.py  datasets.py  carla_bt_runner.py
  deploy/  embedded_bench.py  ros2_node.py  Dockerfile
  tests/  test_all.py  test_datasets.py
  figures/ localization_results.png  bt_episode.png  closed_loop.png
  results/ *.log  *.json
```

Commands: `pip install -r requirements.txt`; `python run_all.py -quick` (tests, E1, E2, E5, E6); `python run_all.py -full` (adds E3, E4, about 15–20 min); `make report` or compile `main.tex` in Overleaf (upload the whole folder).

B Source Code

Listing 1: localization.py

```python
"""
HD-map localization: NDT scan alignment + particle filter (NDT-PF).

Components
----------
make_world()          synthetic HD map (road with irregular building walls) + ground-truth drive
NDTMap                Normal Distributions Transform grid built from the pre-mapped HD map points
ndt_align()           Newton scan-to-map alignment (local refinement)
NDTParticleFilter     particle filter whose measurement likelihood is the NDT score, plus
                      "score-gated NDT injection" and score-triggered cloud inflation (recovery)

Only numpy is required.
"""
import numpy as np


def wrap(a):
    return (a + np.pi) % (2 * np.pi) - np.pi


def transform_batch(poses, pts):
    """poses (P,3), pts (N,2) in vehicle frame -> (P,N,2) in map frame."""
    c = np.cos(poses[:, 2])[:, None]
    s = np.sin(poses[:, 2])[:, None]
    x = c * pts[None, :, 0] - s * pts[None, :, 1] + poses[:, 0:1]
    y = s * pts[None, :, 0] + c * pts[None, :, 1] + poses[:, 1:2]
    return np.stack([x, y], axis=-1)


# ------------------------------------------------------------------------- world
def make_world(seed=0, step=0.5):
    """Closed road with irregular building setbacks. Returns map points and the drive path."""
    rng = np.random.default_rng(seed)
    phi = np.linspace(0, 2 * np.pi, 4000, endpoint=False)
    r = 60 + 12 * np.sin(3 * phi) + 6 * np.cos(2 * phi)
    xy = np.c_[r * np.cos(phi), r * np.sin(phi)]
    seg = np.linalg.norm(np.diff(np.vstack([xy, xy[:1]]), axis=0), axis=1)
    arc = np.concatenate([[0], np.cumsum(seg)])
    total = arc[-1]
    s_grid = np.arange(0, total, step)
    px = np.interp(s_grid, arc, np.append(xy[:, 0], xy[0, 0]))
    py = np.interp(s_grid, arc, np.append(xy[:, 1], xy[0, 1]))
    path = np.c_[px, py]
    d = np.gradient(path, axis=0)
    heading = np.unwrap(np.arctan2(d[:, 1], d[:, 0]))
    normal = np.c_[-np.sin(heading), np.cos(heading)]

    block = 12.0  # metres per building block, each with its own setback
    n_blocks = int(total // block) + 1
    pts = []
    for side in (+1, -1):
        offs = rng.uniform(7.0, 15.0, n_blocks + 1)
        for i, s in enumerate(s_grid):
            k = int(s // block)
            off = offs[k]
            pts.append(path[i] + side * off * normal[i])
            if i > 0 and int(s_grid[i - 1] // block) != k:  # facade connecting two setbacks
                for o in np.arange(min(offs[k - 1], off), max(offs[k - 1], off), step):
                    pts.append(path[i] + side * o * normal[i])
    # sparse poles near the kerb add extra structure
```

```
    for i in range(0, len(s_grid), 14):
        pts.append(path[i] + rng.choice([-1, 1]) * 4.5 * normal[i])
    pts = np.array(pts)
    return pts, path, heading, total


def simulate_scan(pose, map_pts, rng, n=80, rng_max=25.0, sigma=0.10, outlier_frac=0.10):
    """Lidar-like scan: random visible map points within range, in the vehicle frame."""
    d = map_pts - pose[:2]
    dist = np.linalg.norm(d, axis=1)
    cand = np.where(dist < rng_max)[0]
    if len(cand) < 5:
        return np.zeros((0, 2))
    sel = rng.choice(cand, size=min(n, len(cand)), replace=False)
    c, s = np.cos(-pose[2]), np.sin(-pose[2])
    R = np.array([[c, -s], [s, c]])
    local = (d[sel] @ R.T) + rng.normal(0, sigma, (len(sel), 2))
    k = int(outlier_frac * len(local))
    if k:
        local[:k] = rng.uniform(-rng_max, rng_max, (k, 2))
    return local


# ---------------------------------------------------------------------------- NDT
class NDTMap:
    """Grid of Gaussians (mean, inverse covariance) fitted to the HD map points."""

    def __init__(self, pts, cell=4.0, min_pts=4, reg=0.15):
        self.cell = cell
        self.origin = pts.min(0) - cell
        self.dims = np.ceil((pts.max(0) + cell - self.origin) / cell).astype(int) + 1
        self.mean = np.zeros((*self.dims, 2))
        self.icov = np.zeros((*self.dims, 2, 2))
        self.valid = np.zeros(self.dims, bool)
        idx = np.floor((pts - self.origin) / cell).astype(int)
        flat = idx[:, 0] * self.dims[1] + idx[:, 1]
        order = np.argsort(flat)
        flat_s, pts_s = flat[order], pts[order]
        bounds = np.flatnonzero(np.diff(flat_s)) + 1
        for chunk, f in zip(np.split(pts_s, bounds), flat_s[np.r_[0, bounds]]):
            if len(chunk) < min_pts:
                continue
            ix, iy = divmod(int(f), int(self.dims[1]))
            mu = chunk.mean(0)
            C = np.cov(chunk.T) + reg * np.eye(2)  # reg keeps wall cells from being degenerate
            self.mean[ix, iy] = mu
            self.icov[ix, iy] = np.linalg.inv(C)
            self.valid[ix, iy] = True

    def score(self, poses, scan):
        """Sum over scan points of exp(-0.5 d^T C^-1 d) for each pose -> (P,)."""
        if len(scan) == 0:
            return np.zeros(len(poses))
        w = transform_batch(poses, scan)
        idx = np.floor((w - self.origin) / self.cell).astype(int)
        inb = (idx >= 0).all(-1) & (idx[..., 0] < self.dims[0]) & (idx[..., 1] < self.dims[1])
        ix = np.clip(idx[..., 0], 0, self.dims[0] - 1)
        iy = np.clip(idx[..., 1], 0, self.dims[1] - 1)
        ok = inb & self.valid[ix, iy]
        d = w - self.mean[ix, iy]
        m = np.einsum("pni,pnij,pnj->pn", d, self.icov[ix, iy], d)
        return np.where(ok, np.exp(-0.5 * m), 0.0).sum(1)


def ndt_align(ndt, scan, init, iters=10, damping=1e-2, max_step=(1.0, 1.0, 0.1)):
    """Maximise the NDT score with damped Newton steps (finite-difference derivatives)."""
    pose = np.array(init, float)
    eps = np.array([0.05, 0.05, 0.005])
    max_step = np.array(max_step)
    for _ in range(iters):
        # build batch: centre, +-e_i, and four combos per pair
        batch = [pose]
        for i in range(3):
            for sgn in (+1, -1):
                p = pose.copy(); p[i] += sgn * eps[i]; batch.append(p)
        pairs = [(0, 1), (0, 2), (1, 2)]
        for i, j in pairs:
            for si, sj in ((1, 1), (1, -1), (-1, 1), (-1, -1)):
                p = pose.copy(); p[i] += si * eps[i]; p[j] += sj * eps[j]; batch.append(p)
        f = ndt.score(np.array(batch), scan)
        f0 = f[0]
        g = np.array([(f[1 + 2 * i] - f[2 + 2 * i]) / (2 * eps[i]) for i in range(3)])
        H = np.zeros((3, 3))
        for i in range(3):
            H[i, i] = (f[1 + 2 * i] - 2 * f0 + f[2 + 2 * i]) / eps[i] ** 2
```

```
        for k, (i, j) in enumerate(pairs):
            b = 7 + 4 * k
            H[i, j] = H[j, i] = (f[b] - f[b + 1] - f[b + 2] + f[b + 3]) / (4 * eps[i] * eps[j])
        # maximise f: need negative-definite H; otherwise fall back to gradient ascent
        Hn = -H + damping * np.eye(3) * max(1.0, np.abs(np.diag(H)).max())
        try:
            if np.all(np.linalg.eigvalsh(Hn) > 0):
                step = np.linalg.solve(Hn, g)
            else:
                step = g / (np.abs(g).max() + 1e-9) * max_step * 0.3
        except np.linalg.LinAlgError:
            break
        step = np.clip(step, -max_step, max_step)
        new = pose + step
        if ndt.score(new[None], scan)[0] < f0:  # backtrack once
            new = pose + 0.3 * step
            if ndt.score(new[None], scan)[0] < f0:
                break
        pose = new
        if np.abs(step).max() < 1e-3:
            break
    pose[2] = wrap(pose[2])
    return pose, ndt.score(pose[None], scan)[0]


# ------------------------------------------------------------------------- PF
class NDTParticleFilter:
    """
    Particle filter with an NDT likelihood.

    Two extras beyond a textbook PF (the project's own contribution):
      * score-gated NDT injection: after each update, run a Newton NDT alignment from the weighted
        mean; if it scores clearly better than the mean, the worst particles are re-seeded around
        the refined pose (and the estimate is the refined pose). This fixes particle-set
        impoverishment/quantisation error without trusting NDT blindly (it only acts when the
        score confirms it).
      * score-triggered inflation: if the normalised score stays low for several steps (map
        mismatch, drift, kidnapping) the process noise is inflated so the cloud can re-acquire.
    """

    def __init__(self, ndt, init_pose, init_std=(2.0, 2.0, 0.15), n=300, seed=0,
                 use_injection=True, temper=0.5):
        self.ndt, self.n, self.rng = ndt, n, np.random.default_rng(seed)
        self.p = np.array(init_pose) + self.rng.normal(0, 1, (n, 3)) * np.array(init_std)
        self.w = np.full(n, 1.0 / n)
        self.use_injection, self.temper = use_injection, temper
        self.low_count, self.inflate = 0, 1.0
        self.est = np.array(init_pose, float)
        self.last_score = 0.0

    def predict(self, v, w, dt):
        s = self.inflate
        vn = v + self.rng.normal(0, 0.20 * s, self.n)
        wn = w + self.rng.normal(0, 0.04 * s, self.n)
        th = self.p[:, 2]
        self.p[:, 0] += vn * dt * np.cos(th + 0.5 * wn * dt)
        self.p[:, 1] += vn * dt * np.sin(th + 0.5 * wn * dt)
        self.p[:, 2] = wrap(th + wn * dt)

    def update(self, scan, n_ref=None):
        sc = self.ndt.score(self.p, scan)
        n = max(len(scan), 1)
        norm = sc / n  # 0..1 : fraction of points explained
        logw = self.temper * np.log(sc + 1e-3 * n) + np.log(self.w + 1e-300)
        logw -= logw.max()
        self.w = np.exp(logw); self.w /= self.w.sum()
        ess = 1.0 / np.sum(self.w ** 2)
        if ess < self.n / 2:
            self._resample()
        self.est = self._mean()
        self.last_score = float(self.ndt.score(self.est[None], scan)[0] / n)

        if self.use_injection:
            ref, rs = ndt_align(self.ndt, scan, self.est, iters=6)
            ref_norm = rs / n
            if ref_norm > self.last_score + 0.02 and np.linalg.norm(ref[:2] - self.est[:2]) < 3.0:
                k = self.n // 5
                worst = np.argsort(self.w)[:k]
                self.p[worst] = ref + self.rng.normal(0, 1, (k, 3)) * np.array([0.25, 0.25, 0.02])
                self.w[worst] = self.w.max()
                self.w /= self.w.sum()
                self.est = ref
                self.last_score = float(ref_norm)
        # recovery logic
        if self.last_score < 0.25:
```

```
            self.low_count += 1
        else:
            self.low_count = 0
        self.inflate = 3.0 if self.low_count >= 5 else 1.0
        return self.est

    def _resample(self):
        pos = (self.rng.random() + np.arange(self.n)) / self.n
        idx = np.minimum(np.searchsorted(np.cumsum(self.w), pos), self.n - 1)
        self.p = self.p[idx]
        self.w = np.full(self.n, 1.0 / self.n)

    def _mean(self):
        x = np.sum(self.w * self.p[:, 0]); y = np.sum(self.w * self.p[:, 1])
        th = np.arctan2(np.sum(self.w * np.sin(self.p[:, 2])), np.sum(self.w * np.cos(self.p[:, 2])))
        return np.array([x, y, th])
```

Listing 2: run_localization.py

```
"""
Localization experiment: dead reckoning vs NDT-only vs plain PF vs NDT-PF (ours), over several seeds.
Run:  python run_localization.py [--plot]
"""
import os, sys
import numpy as np
from localization import (make_world, simulate_scan, NDTMap, ndt_align,
                          NDTParticleFilter, wrap)

DT, SPEED = 0.1, 5.0  # 5 m/s * 0.1 s = 0.5 m = map path resolution


def drive(seed, map_pts, path, heading, methods=("dr", "ndt", "pf", "ndtpf"),
          init_offset=(1.0, -1.0, 0.08), init_std=(2.0, 2.0, 0.15), faults=None):
    faults = faults or {}
    world_pts = faults.get("world_pts", map_pts)          # what the lidar really sees (may differ from the HD map)
    rng = np.random.default_rng(1000 + seed)
    ndt = NDTMap(map_pts)
    step = 0.5
    stride = max(1, int(round(SPEED * DT / step)))
    idxs = np.arange(0, len(path), stride)
    truth = np.c_[path[idxs], heading[idxs]]
    yaw_rate = wrap(np.diff(truth[:, 2], prepend=truth[0, 2])) / DT
    yaw_rate[0] = 0

    init = truth[0] + np.array(init_offset)  # imperfect initial guess
    est = {m: [init.copy()] for m in methods}
    pf = NDTParticleFilter(ndt, init, init_std=init_std, n=300, seed=seed, use_injection=False)
    ours = NDTParticleFilter(ndt, init, init_std=init_std, n=300, seed=seed, use_injection=True)
    dr, nd = init.copy(), init.copy()
    bias = faults.get("gyro_bias", 0.01)  # systematic gyro bias (rad/s) -> dead reckoning drifts
    scores = []

    for k in range(1, len(truth)):
        v = SPEED * 1.02 + rng.normal(0, 0.10)
        w = yaw_rate[k] + bias + rng.normal(0, 0.02)
        for pose in (dr, nd):
            pose[0] += v * DT * np.cos(pose[2] + 0.5 * w * DT)
            pose[1] += v * DT * np.sin(pose[2] + 0.5 * w * DT)
            pose[2] = wrap(pose[2] + w * DT)
        t_now = k * DT
        bo = faults.get("blackout")
        if bo and bo[0] <= t_now < bo[1]:
            scan = np.zeros((0, 2))                                  # sensor dropout
        else:
            scan = simulate_scan(truth[k], world_pts, rng, sigma=faults.get("sigma", 0.10),
                                 outlier_frac=faults.get("outlier_frac", 0.10))
        if "ndt" in methods:
            nd, _ = ndt_align(ndt, scan, nd, iters=8)
        if "pf" in methods:
            pf.predict(v, w, DT); pf.update(scan)
        if "ndtpf" in methods:
            ours.predict(v, w, DT); ours.update(scan)
        scores.append(ours.last_score)
        est["dr"].append(dr.copy()); est["ndt"].append(nd.copy())
        est["pf"].append(pf.est.copy()); est["ndtpf"].append(ours.est.copy())
    out = {m: np.array(est[m]) for m in methods}
    out["scores"] = np.array(scores)
    return truth, out


def errors(truth, e):
    pos = np.linalg.norm(e[:, :2] - truth[:, :2], axis=1)
    yaw = np.abs(wrap(e[:, 2] - truth[:, 2]))
    return pos, np.degrees(yaw)
```

```

if __name__ == "__main__":
    map_pts, path, heading, total = make_world(seed=0)
    print(f"HD map: {len(map_pts)} points, route length {total:.0f} m")
    names = {"dr": "Dead reckoning", "ndt": "NDT only", "pf": "PF (NDT likelihood)",
             "ndtpf": "NDT-PF (ours)"}
    scenarios = {"nominal init (1.4 m, 5 deg off)": dict(init_offset=(1.0, -1.0, 0.08), init_std=(2.0, 2.0, 0.15)),
                 "hard init (5 m, 20 deg off)": dict(init_offset=(4.0, -3.0, 0.35), init_std=(4.0, 4.0, 0.35))}
    n_seeds = 5
    last = None
    for sname, kw in scenarios.items():
        agg = {m: [] for m in names}
        for seed in range(n_seeds):
            truth, est = drive(seed, map_pts, path, heading, **kw)
            if last is None:
                last = (truth, est)
            for m in names:
                pos, yaw = errors(truth, est[m])
                agg[m].append((np.sqrt((pos ** 2).mean()), np.percentile(pos, 95), pos.max(),
                               np.sqrt((yaw ** 2).mean()), float(pos[-50:].mean() < 1.0)))
        print(f"\nScenario: {sname}  ({n_seeds} seeds, mean of per-run metrics)")
        print(f"{'method':24s} {'RMSE pos [m]':>13s} {'p95 [m]':>9s} {'max [m]':>9s} {'RMSE yaw [deg]':>15s} {'converged':>10s}")
        for m, label in names.items():
            a = np.array(agg[m]).mean(0)
            print(f"{label:24s} {a[0]:13.2f} {a[1]:9.2f} {a[2]:9.2f} {a[3]:15.2f} {a[4]*100:9.0f}%")

    if "--plot" in sys.argv:
        import matplotlib; matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        truth, est = last
        fig, ax = plt.subplots(1, 2, figsize=(13, 5))
        ax[0].scatter(map_pts[::3, 0], map_pts[::3, 1], s=1, c="lightgray")
        ax[0].plot(truth[:, 0], truth[:, 1], "k", lw=1, label="truth")
        ax[0].plot(est["dr"][:, 0], est["dr"][:, 1], "r--", lw=1, label="dead reckoning")
        ax[0].plot(est["ndtpf"][:, 0], est["ndtpf"][:, 1], "g", lw=1, label="NDT-PF")
        ax[0].set_aspect("equal"); ax[0].legend(); ax[0].set_title("Trajectory over HD map")
        t = np.arange(len(truth)) * DT
        for m, c in (("dr", "r"), ("ndt", "b"), ("pf", "orange"), ("ndtpf", "g")):
            ax[1].plot(t, errors(truth, est[m])[0], c, label=names[m])
        ax[1].set_yscale("log"); ax[1].set_xlabel("time [s]"); ax[1].set_ylabel("position error [m]")
        ax[1].legend(); ax[1].set_title("Localization error")
        plt.tight_layout(); os.makedirs("figures", exist_ok=True); plt.savefig("figures/localization_results.png", dpi=120)
        print("saved figures/localization_results.png")
```

Listing 3: behavior_tree.py

```
"""
Behavior-tree decision making for overtaking, roundabout entry and yielding.

Part 1: a small BT library (Selector / Sequence with optional memory / Condition / Action / Guard).
Part 2: a 1-D route world (lane 0 = ego lane, lane 1 = oncoming lane) and the driving tree.
Part 3: randomised batch evaluation (python behavior_tree.py).
"""
import os, sys
from enum import Enum
import numpy as np


# ---------------------------------------------------------------- BT library
class Status(Enum):
    SUCCESS, FAILURE, RUNNING = 1, 2, 3


class Node:
    def __init__(self, name):
        self.name = name

    def tick(self, bb):
        raise NotImplementedError

    def reset(self):
        pass


class Condition(Node):
    def __init__(self, name, fn):
        super().__init__(name); self.fn = fn

    def tick(self, bb):
        return Status.SUCCESS if self.fn(bb) else Status.FAILURE


class Action(Node):
```

```
    """fn(bb) -> Status. Writes its commands (target speed / lane) into the blackboard."""
    def __init__(self, name, fn):
        super().__init__(name); self.fn = fn

    def tick(self, bb):
        st = self.fn(bb)
        if st == Status.RUNNING:
            bb["active"].append(self.name)
        return st


class Sequence(Node):
    """memory=True: resumes at the running child (used for committed manoeuvres)."""
    def __init__(self, name, children, memory=False):
        super().__init__(name); self.children, self.memory, self.i = children, memory, 0

    def tick(self, bb):
        start = self.i if self.memory else 0
        for k in range(start, len(self.children)):
            st = self.children[k].tick(bb)
            if st == Status.RUNNING:
                self.i = k
                return st
            if st == Status.FAILURE:
                self.reset(); return st
        self.reset()
        return Status.SUCCESS

    def reset(self):
        self.i = 0
        for c in self.children:
            c.reset()


class Selector(Node):
    """Reactive priority selector; lower-priority branches are reset when a higher one takes over."""
    def __init__(self, name, children):
        super().__init__(name); self.children = children

    def tick(self, bb):
        for k, c in enumerate(self.children):
            st = c.tick(bb)
            if st != Status.FAILURE:
                for other in self.children[k + 1:]:
                    other.reset()
                return st
        return Status.FAILURE

    def reset(self):
        for c in self.children:
            c.reset()


class Guard(Node):
    """Runs child only while cond holds; otherwise FAILURE (abort condition for a manoeuvre)."""
    def __init__(self, name, cond, child):
        super().__init__(name); self.cond, self.child = cond, child

    def tick(self, bb):
        if not self.cond(bb):
            self.child.reset(); return Status.FAILURE
        return self.child.tick(bb)

    def reset(self):
        self.child.reset()


# ------------------------------------------------------------------ world
V_DES, A_MAX, B_COMF = 12.0, 2.5, 3.0
CAR_LEN = 4.5
ROUND_ENTRY, ROUND_END = 300.0, 345.0
YIELD_LINE = ROUND_ENTRY - 3.0   # stop line (ego front)
ROUTE_END = 420.0


class World:
    def __init__(self, seed, params=None):
        r = np.random.default_rng(seed)
        self.t = 0.0
        self.s, self.v, self.lat = 0.0, 0.0, 0.0          # lat: 0 = lane 0, 1 = lane 1
        self.lead_s = r.uniform(30, 60)
        self.lead_v = r.uniform(3.0, 7.0)
        self.lead_v0 = self.lead_v                        # lead's desired speed (it follows ego with IDM if ego is ahead)
        self.onc_s = r.uniform(180, 320)                  # oncoming car in lane 1
        self.onc_v = -r.uniform(9.0, 13.0)
```

```
        n_cv = r.integers(2, 5)                          # circulating vehicles: times they pass conflict point
        self.cv_times = np.sort(r.uniform(18, 60, n_cv))
        if params is not None:                           # explicit scenario (used by formal/exhaustive checks)
            self.lead_s, self.lead_v = params["lead_s"], params["lead_v"]
            self.lead_v0 = self.lead_v
            self.onc_s, self.onc_v = params["onc_s"], -abs(params["onc_v"])
            self.cv_times = np.array(sorted(params["cv_times"]), float)
        self.collisions = 0
        self.min_gap = 1e9
        self.yield_wait = 0.0
        self.overtakes = 0

    # tick the world with commands from the BT
    def step(self, cmd, dt=0.1):
        tv, tl = cmd["speed"], cmd["lane"]
        a = np.clip((tv - self.v) / 0.6, -B_COMF * 1.8, A_MAX)
        self.v = max(0.0, self.v + a * dt)
        self.s += self.v * dt
        self.lat += np.clip(tl - self.lat, -dt / 2.0, dt / 2.0)   # 2 s per lane change
        # lead car: IDM follower when ego is ahead of it in lane 0 (so it never rear-ends a stopped ego)
        gap = self.s - self.lead_s - CAR_LEN
        if self.lat < 0.9 and gap > -CAR_LEN:
            dv = self.lead_v - self.v
            s_star = 2.0 + self.lead_v * 1.2 + self.lead_v * dv / (2 * np.sqrt(2.0 * 3.0))
            acc = 2.0 * (1 - (self.lead_v / self.lead_v0) ** 4 - (s_star / max(gap, 0.1)) ** 2)
            self.lead_v = max(0.0, self.lead_v + np.clip(acc, -6.0, 2.0) * dt)
        self.lead_s += self.lead_v * dt
        self.onc_s += self.onc_v * dt
        self.t += dt
        # safety metrics
        in0, in1 = self.lat < 0.9, self.lat > 0.1
        if in0 and abs(self.lead_s - self.s) < CAR_LEN:
            self.collisions += 1
        if in1 and abs(self.onc_s - self.s) < CAR_LEN:
            self.collisions += 1
        if in0 and self.lead_s > self.s:
            self.min_gap = min(self.min_gap, self.lead_s - self.s - CAR_LEN)
        if in1 and self.onc_s > self.s:
            self.min_gap = min(self.min_gap, self.onc_s - self.s - CAR_LEN)
        # collision with circulating traffic: ego inside conflict zone while a cv passes
        if ROUND_ENTRY - 1.0 < self.s < ROUND_ENTRY + CAR_LEN:   # physical conflict zone
            for tc in self.cv_times:
                if abs(self.t - tc) < 1.0:
                    self.collisions += 1
        if self.v < 0.3 and YIELD_LINE - 8 < self.s < ROUND_ENTRY:
            self.yield_wait += dt


def bb_from_world(w):
    return dict(w=w, active=[])


# ------------------------------------------------------------------ conditions / actions
def lead_gap(bb):
    w = bb["w"]
    return w.lead_s - w.s - CAR_LEN if w.lat < 0.9 and w.lead_s > w.s else 1e9


def blocked_by_slow_lead(bb):
    """Slow lead close ahead AND the whole pass (incl. return to lane 0) finishes before the roundabout
    approach zone starts -- the roundabout branch has priority and must never interrupt a pass."""
    w = bb["w"]
    if bb.get("degraded"):                       # perception supervisor flagged a fault: no overtaking
        return False
    if not (lead_gap(bb) < 28 and w.lead_v < 0.75 * V_DES):
        return False
    _, d_ego = predict_pass(w)
    return w.s + d_ego + 15.0 < YIELD_LINE - 60.0


def raw_lead_gap(w):
    return w.lead_s - w.s - CAR_LEN


def predict_pass(w):
    """Kinematic prediction of the overtake: returns (duration, ego distance travelled).
    Ego accelerates at A_MAX towards V_DES, lead keeps its speed; done when the ego is
    2 car lengths + 12 m ahead, plus 2 s for the lane change back."""
    t, se, ve, sl = 0.0, w.s, w.v, w.lead_s
    while se < sl + 2 * CAR_LEN + 12 and t < 40:
        ve = min(V_DES, ve + A_MAX * 0.1)
        se += ve * 0.1; sl += w.lead_v * 0.1; t += 0.1
    t += 2.0
    return t, se - w.s + 2.0 * V_DES

```

```python

def overtake_safe(bb, k=1.3, margin=40.0):
    """Oncoming car must stay clear for the whole predicted manoeuvre (+30% time margin, +40 m) -- start
    condition. The abort test uses k=1.0, margin=15 m: hysteresis, so the tree cannot chatter between lanes."""
    w = bb["w"]
    gap = w.onc_s - w.s
    if gap < -(2 * CAR_LEN + 5.0):                      # oncoming car fully past us (bumper clearance + margin)
        return True
    t, d_ego = predict_pass(w)
    return gap > 0 and gap - d_ego - abs(w.onc_v) * k * t > margin


def merge_back_safe(w):
    """Can we return to lane 0 behind the lead without hitting it? (2.5 s lane change + 8 m reserve)"""
    return raw_lead_gap(w) > max(w.v - w.lead_v, 0.0) * 2.5 + 8.0


def oncoming_unsafe_abort(bb):
    """Abort only while still behind the lead AND merging back is itself safe; otherwise we are committed."""
    w = bb["w"]
    behind = w.s < w.lead_s - CAR_LEN - 1.0
    return behind and not overtake_safe(bb, k=1.0, margin=15.0) and merge_back_safe(w)


def go_left(bb):
    bb["cmd"] = dict(speed=min(V_DES, bb["w"].lead_v + 5.0 + max(0, lead_gap(bb)) * 0.0 + 4.0), lane=1)
    return Status.SUCCESS if bb["w"].lat > 0.95 else Status.RUNNING


def pass_lead(bb):
    w = bb["w"]
    bb["cmd"] = dict(speed=V_DES, lane=1)
    return Status.SUCCESS if w.s > w.lead_s + CAR_LEN + 8 else Status.RUNNING


def return_right(bb):
    w = bb["w"]
    bb["cmd"] = dict(speed=V_DES, lane=0)
    if w.lat < 0.05:
        w.overtakes += 1
        return Status.SUCCESS
    return Status.RUNNING


def approaching_roundabout(bb):
    w = bb["w"]
    in_zone = 0 < YIELD_LINE - w.s < 60 or YIELD_LINE < w.s < ROUND_END
    return in_zone and w.lat < 0.02          # never preempt a lane change / overtake in progress


def before_yield_line(bb):
    return bb["w"].s < ROUND_ENTRY - 0.5          # commit point: after this we are in the zone


def arrival_time(w):
    """Time to reach the conflict point from current (s, v) assuming accel A_ENTER (kinematic)."""
    d = max(ROUND_ENTRY - w.s, 0.0)
    a = 1.5
    return w.t + (-w.v + np.sqrt(w.v ** 2 + 2 * a * d)) / a


def conflict_ahead(bb):
    """Predicted arrival at the conflict point overlaps (+-3 s) a circulating vehicle."""
    w = bb["w"]
    t_arr = arrival_time(w)
    d = ROUND_ENTRY - w.s
    d_nr = w.v * 0.6 + w.v ** 2 / (2 * 4.5)                  # distance needed to stop (lag + braking)
    win = 4.5 if d < d_nr + 12.0 else 3.0                    # wider window near the point of no return
    return bool(np.any((w.cv_times > t_arr - win) & (w.cv_times < t_arr + win)))


def yield_action(bb):
    """Brake to a stop at the yield line (v = sqrt(2 b d)) and hold until the gap opens."""
    w = bb["w"]
    d = max(YIELD_LINE - w.s - 2.0, 0.0)          # aim 2 m short: actuator lag (0.6 s) makes the car creep forward
    v = min(V_DES * 0.6, np.sqrt(2 * 2.0 * d))
    bb["cmd"] = dict(speed=follow_limit(w, v if d > 0.3 else 0.0), lane=0)
    return Status.RUNNING


def follow_limit(w, v):
    """Car-following speed cap applied to every longitudinal command (not only in cruise)."""
    g = lead_gap_w(w)
    if g < 60:
        v = min(v, max(w.lead_v + 0.4 * (g - (6 + 1.2 * w.v)), 0.0))
```

```
    return v


def lead_gap_w(w):
    return w.lead_s - w.s - CAR_LEN if w.lat < 0.9 and w.lead_s > w.s else 1e9


def enter_roundabout(bb):
    w = bb["w"]
    bb["cmd"] = dict(speed=follow_limit(w, 6.0 if w.s < ROUND_END else V_DES), lane=0)
    return Status.RUNNING


def cruise(bb):
    """Adaptive cruise: free-flow V_DES, or follow the lead with a time-gap controller."""
    w = bb["w"]
    g = lead_gap(bb)
    v = V_DES
    if g < 60:
        v = min(V_DES, w.lead_v + 0.4 * (g - (6 + 1.2 * w.v)))
    bb["cmd"] = dict(speed=max(v, 0.0), lane=0)
    return Status.RUNNING


def build_tree():
    overtake = Guard("overtake_guard", lambda bb: not oncoming_unsafe_abort(bb),
                     Sequence("overtake", [
                         Condition("blocked_by_slow_lead", blocked_by_slow_lead),
                         Condition("overtake_safe", overtake_safe),
                         Action("go_left", go_left),
                         Action("pass_lead", pass_lead),
                         Action("return_right", return_right)], memory=True))
    roundabout = Sequence("roundabout", [
        Condition("approaching_roundabout", approaching_roundabout),
        Selector("yield_or_enter", [
            Sequence("yield", [Condition("before_yield_line", before_yield_line),
                               Condition("conflict_ahead", conflict_ahead),
                               Action("yield", yield_action)]),
            Action("enter_roundabout", enter_roundabout)])])
    return Selector("root", [roundabout, overtake, Action("cruise", cruise)])


# ---------------------------------------------------------------- simulation
def run_episode(seed, trace=False, params=None, monitor=None, tree=None):
    w, tree = World(seed, params), (tree or build_tree())
    log = []
    while w.s < ROUTE_END and w.t < 150:
        bb = bb_from_world(w)
        bb["cmd"] = dict(speed=0.0, lane=0)
        tree.tick(bb)
        w.step(bb["cmd"])
        if monitor:
            monitor(w, bb)
        if trace:
            log.append((w.t, w.s, w.v, w.lat, bb["active"][-1] if bb["active"] else "-"))
    done = w.s >= ROUTE_END
    return dict(done=done, time=w.t, collisions=w.collisions, min_gap=w.min_gap,
                overtakes=w.overtakes, yield_wait=w.yield_wait), log


if __name__ == "__main__":
    n = 300
    res = [run_episode(s)[0] for s in range(n)]
    ok = np.array([r["done"] for r in res]); col = np.array([r["collisions"] > 0 for r in res])
    print(f"{n} randomised episodes (random lead speed/position, oncoming car, circulating traffic)")
    print(f"  completed route        : {ok.mean()*100:.1f}%")
    print(f"  episodes with collision: {col.mean()*100:.1f}%")
    print(f"  overtakes performed    : {np.mean([r['overtakes'] for r in res]):.2f} per episode "
          f"({np.mean([r['overtakes'] > 0 for r in res])*100:.0f}% of episodes)")
    print(f"  mean route time        : {np.mean([r['time'] for r in res if r['done']]):.1f} s")
    print(f"  min gap (median / 5th) : {np.median([r['min_gap'] for r in res]):.1f} m / "
          f"{np.percentile([r['min_gap'] for r in res], 5):.1f} m")
    print(f"  episodes that had to wait at yield line: {np.mean([r['yield_wait'] > 0.5 for r in res])*100:.0f}%")
    if "--plot" in sys.argv:
        import matplotlib; matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        _, log = run_episode(3, trace=True)
        L = np.array([(a, b, c, d) for a, b, c, d, _ in log]); names = [e for *_, e in log]
        fig, ax = plt.subplots(3, 1, figsize=(9, 7), sharex=True)
        ax[0].plot(L[:, 0], L[:, 1]); ax[0].set_ylabel("s [m]")
        ax[1].plot(L[:, 0], L[:, 2]); ax[1].set_ylabel("speed [m/s]")
        uniq = list(dict.fromkeys(names))
        ax[2].step(L[:, 0], [uniq.index(n_) for n_ in names]); ax[2].set_yticks(range(len(uniq)))
        ax[2].set_yticklabels(uniq); ax[2].set_xlabel("time [s]"); ax[2].set_ylabel("active leaf")
        plt.tight_layout(); os.makedirs("figures", exist_ok=True); plt.savefig("figures/bt_episode.png", dpi=120); print("saved
```

```
    figures/bt_episode.png")
```

Listing 4: verify_bt.py

```
"""
Verification of the driving behaviour tree.

(1) Static / structural checks on the tree (totality, every leaf reachable, depth).
(2) Runtime monitors for safety properties (signal-temporal-logic style, evaluated every tick).
(3) Bounded exhaustive verification: the scenario parameter space is discretised into a grid and
    EVERY grid point is simulated against the monitors (deterministic world => complete for the grid).
    This is verification relative to the grid and the 1-D world model -- not a proof for the real world.
Run: python verify_bt.py
"""
import itertools, sys, time
import numpy as np
import behavior_tree as bt

TTC_MIN = 3.0   # property P2: never in the oncoming lane with time-to-collision below this


class Monitor:
    """Safety properties, G(...) = must hold at every tick."""
    def __init__(self):
        self.viol = {}
        self.prev_lat, self.max_active = 0.0, 0

    def __call__(self, w, bb):
        def bad(name):
            self.viol.setdefault(name, w.t)
        # P1: G(no collision)  (recorded by world)
        if w.collisions:
            bad("P1_no_collision")
        # P2: G(in oncoming lane -> TTC to oncoming car >= TTC_MIN)
        if w.lat > 0.1 and w.onc_s > w.s:
            ttc = (w.onc_s - w.s - bt.CAR_LEN) / max(w.v + abs(w.onc_v), 1e-3)
            if ttc < TTC_MIN:
                bad("P2_oncoming_TTC")
        # P3: G(speed in roundabout zone <= 8 m/s)
        if bt.ROUND_ENTRY - 1 < w.s < bt.ROUND_END and w.v > 8.0:
            bad("P3_roundabout_speed")
        # P4: G(no lane change inside/near roundabout)
        if w.s > bt.ROUND_ENTRY - 30 and w.s < bt.ROUND_END and abs(w.lat) > 0.02:
            bad("P4_no_lane_change_near_roundabout")
        # P5: G(never stationary in the conflict zone while a circulating car arrives)  (subset of P1, explicit)
        # P6: G(exactly <=1 active leaf per tick)
        if len(set(bb["active"])) > 1:
            bad("P6_single_active_leaf")


def structural_checks():
    tree = bt.build_tree()
    leaves, depth = [], [0]

    def walk(n, d=0):
        depth[0] = max(depth[0], d)
        kids = getattr(n, "children", None) or ([n.child] if hasattr(n, "child") else [])
        if not kids:
            leaves.append(n.name)
        for k in kids:
            walk(k, d + 1)
    walk(tree)
    out = {"leaves": leaves, "depth": depth[0]}
    # totality: root's last child is an Action that never returns FAILURE
    out["root_total"] = tree.children[-1].name == "cruise"
    return out


def run_grid(grid, label):
    t0 = time.time(); n = 0; fails = {}; incomplete = 0; examples = {}
    for combo in itertools.product(*grid.values()):
        p = dict(zip(grid.keys(), combo))
        p["cv_times"] = p.pop("cv", ())
        m = Monitor()
        res, _ = bt.run_episode(0, params=p, monitor=m)
        n += 1
        if not res["done"]:
            incomplete += 1
            examples.setdefault("LIVENESS", p)
        for k in m.viol:
            fails[k] = fails.get(k, 0) + 1
            examples.setdefault(k, p)
    print(f"\n[{label}] {n} scenarios in {time.time()-t0:.0f}s")
    print("  violations:", fails if fails else "none", "| incomplete routes:", incomplete)
```

```
    for k, v in examples.items():
        print("  counterexample", k, {a: (np.round(b, 1).tolist() if hasattr(b, '__len__') else b) for a, b in v.items()})
    return n, fails, incomplete


if __name__ == "__main__":
    st = structural_checks()
    print("structure:", st)
    # sweep A: overtaking sub-problem, fine grid, no roundabout traffic
    gridA = dict(lead_s=range(20, 71, 5), lead_v=[3, 4, 5, 6, 7],
                 onc_s=range(120, 361, 20), onc_v=[9, 10, 11, 12, 13], cv=[()])
    # sweep B: roundabout yielding: all subsets of <=3 circulating-car passage times (coarse), fixed others
    slots = list(range(20, 61, 4))
    cvs = [c for r in range(0, 4) for c in itertools.combinations(slots, r)]
    gridB = dict(lead_s=[30, 50], lead_v=[4, 6], onc_s=[200, 340], onc_v=[11], cv=cvs)
    run_grid(gridA, "A overtaking")
    run_grid(gridB, "B roundabout yield")
```

Listing 5: planning.py

```
"""
Motion planning in the Frenet frame.

1. Space-time lattice: a 3-D lattice of terminal conditions (lateral offset l_f x terminal speed v_f x
   horizon T, plus "follow-the-lead" terminals) is expanded with quintic (lateral) / quartic or quintic
   (longitudinal) jerk-optimal polynomials, checked against moving obstacles in (s, l, t) space-time,
   and ranked with a cost function (Werling et al. 2010 style).
2. Trajectory optimisation: the winning lattice trajectory is re-optimised as a bounded least-squares
   QP (minimum jerk + deviation from the lattice solution, with obstacle-induced corridor bounds on
   l(t) and s(t)). Dynamic limits are checked afterwards and reported.
"""
from dataclasses import dataclass, field
import numpy as np
from scipy.optimize import lsq_linear

CAR_L, CAR_W = 4.5, 2.0
A_MAX, A_MIN, A_LAT_MAX, V_MAX = 3.0, -6.0, 3.5, 20.0


@dataclass
class Obstacle:
    s: float; l: float; v: float = 0.0          # constant-speed along the road, fixed lateral position
    def at(self, t): return self.s + self.v * t, self.l


@dataclass
class FState:                                   # Frenet state (time derivatives)
    s: float; sd: float; sdd: float; l: float; ld: float; ldd: float


@dataclass
class Traj:
    t: np.ndarray; s: np.ndarray; l: np.ndarray
    sd: np.ndarray; ld: np.ndarray; sdd: np.ndarray; ldd: np.ndarray
    cost: float = 0.0; kind: str = ""
    def jerk_rms(self):
        j = np.gradient(np.hypot(self.sdd, self.ldd), self.t)
        return float(np.sqrt(np.mean(j ** 2)))


def quintic(x0, v0, a0, x1, v1, a1, T):
    A = np.array([[T ** 3, T ** 4, T ** 5], [3 * T ** 2, 4 * T ** 3, 5 * T ** 4], [6 * T, 12 * T ** 2, 20 * T ** 3]])
    b = np.array([x1 - (x0 + v0 * T + 0.5 * a0 * T ** 2), v1 - (v0 + a0 * T), a1 - a0])
    c3, c4, c5 = np.linalg.solve(A, b)
    return np.array([x0, v0, 0.5 * a0, c3, c4, c5])


def quartic(x0, v0, a0, v1, a1, T):
    A = np.array([[3 * T ** 2, 4 * T ** 3], [6 * T, 12 * T ** 2]])
    b = np.array([v1 - (v0 + a0 * T), a1 - a0])
    c3, c4 = np.linalg.solve(A, b)
    return np.array([x0, v0, 0.5 * a0, c3, c4, 0.0])


def poly_eval(c, t):
    x = c[0] + c[1] * t + c[2] * t ** 2 + c[3] * t ** 3 + c[4] * t ** 4 + c[5] * t ** 5
    v = c[1] + 2 * c[2] * t + 3 * c[3] * t ** 2 + 4 * c[4] * t ** 3 + 5 * c[5] * t ** 4
    a = 2 * c[2] + 6 * c[3] * t + 12 * c[4] * t ** 2 + 20 * c[5] * t ** 3
    return x, v, a


def collides(t, s, l, sd, obstacles, ms=3.0, ml=0.35):
    for o in obstacles:
        so = o.s + o.v * t
```

```
        ds = np.abs(s - so); dl = np.abs(l - o.l)
        # speed-dependent longitudinal buffer (0.5 s headway behind obstacle)
        buf = ms + 0.5 * np.maximum(sd, 0.0) * (s < so)
        if np.any((ds < CAR_L + buf) & (dl < CAR_W + ml)):
            return True
    return False


class LatticePlanner:
    def __init__(self, v_des=12.0, road=(-1.75, 5.25), dt=0.2, kappa_ref=None):
        self.v_des, self.road, self.dt = v_des, road, dt
        self.lf = np.unique(np.r_[np.arange(-0.75, 4.4, 0.875), 0.0, 3.5])     # lateral lattice incl. both lane centres
        self.Ts = (3.0, 4.0, 5.0)
        self.w = dict(jerk=0.1, T=0.4, lat=1.0, speed=1.0, lane=0.6, ldev=0.05)
        self.lane_centres = np.array([0.0, 3.5])

    def candidates(self, st: FState, obstacles):
        out = []
        for T in self.Ts:
            t = np.arange(0, T + 1e-9, self.dt)
            # longitudinal terminals: velocity keeping lattice, plus follow-lead for every obstacle ahead in lane
            lon = []
            for vf in np.linspace(0.0, self.v_des, 7):
                lon.append(("vel", quartic(st.s, st.sd, st.sdd, vf, 0.0, T), None))
            for o in obstacles:
                if o.s > st.s and o.v < self.v_des:
                    sf = o.s + o.v * T - (CAR_L + 6.0 + 1.2 * o.v)
                    if sf > st.s:
                        lon.append(("follow", quintic(st.s, st.sd, st.sdd, sf, o.v, 0.0, T), o))
            for lf in self.lf:
                cl = quintic(st.l, st.ld, st.ldd, lf, 0.0, 0.0, T)
                lv, ldv, lda = poly_eval(cl, t)
                for kind, cs, _ in lon:
                    s, sd, sdd = poly_eval(cs, t)
                    if np.any(sd < -0.1) or np.any(sdd > A_MAX) or np.any(sdd < A_MIN):
                        continue
                    if np.any(np.abs(lda) > 2.5) or np.any(np.abs(sd ** 2 * 0 + lda) > A_LAT_MAX):
                        continue
                    if np.any(lv < self.road[0] + 0.5) or np.any(lv > self.road[1] - 0.5):
                        continue
                    if collides(t, s, lv, sd, obstacles):
                        continue
                    j_l = np.gradient(lda, t); j_s = np.gradient(sdd, t)
                    J = (self.w["jerk"] * (np.trapezoid(j_l ** 2, t) + np.trapezoid(j_s ** 2, t))
                         + self.w["T"] * T
                         + self.w["speed"] * (sd[-1] - self.v_des) ** 2 * 0.25
                         + self.w["lane"] * np.min(np.abs(lf - self.lane_centres)) ** 2
                         + self.w["ldev"] * lf ** 2 * 0.2)
                    out.append(Traj(t, s, lv, sd, ldv, sdd, lda, J, f"{kind}|lf={lf:.2f}|T={T:.0f}"))
        return out

    def plan(self, st: FState, obstacles):
        c = self.candidates(st, obstacles)
        if not c:
            return self.emergency(st), 0
        best = min(c, key=lambda x: x.cost)
        return best, len(c)

    def emergency(self, st: FState):
        T = max(st.sd / 5.0, 1.0)
        t = np.arange(0, T + 1e-9, self.dt)
        cs = quartic(st.s, st.sd, st.sdd, 0.0, 0.0, T)
        s, sd, sdd = poly_eval(cs, t)
        l = np.full_like(t, st.l)
        return Traj(t, s, l, sd, np.zeros_like(t), sdd, np.zeros_like(t), 1e6, "emergency")


# ------------------------------------------------------------------ trajectory optimisation
def optimise_trajectory(tr: Traj, obstacles, w_jerk=1.0, w_dev=0.02, road=(-1.75, 5.25)):
    """
    Bounded least-squares QP on the sample vectors l(t) and s(t):
        min  w_jerk*||D3 x||^2 + w_dev*||x - x_lattice||^2     s.t.  x_lo <= x <= x_hi,  x(0..2) fixed
    Bounds: lateral corridor from obstacle sides chosen by the lattice solution; longitudinal upper
    bound s(t) <= s_obstacle_rear(t) for obstacles that the lattice solution stays behind.
    """
    t = tr.t; n = len(t); dt = t[1] - t[0]
    D3 = np.zeros((n - 3, n))
    for i in range(n - 3):
        D3[i, i:i + 4] = [-1, 3, -3, 1]
    D3 /= dt ** 3
    s_hi = np.full(n, np.inf); l_lo = np.full(n, road[0] + 0.5); l_hi = np.full(n, road[1] - 0.5)
    for o in obstacles:
        so = o.s + o.v * t
        close = np.abs(tr.s - so) < CAR_L + 12
        if not close.any():
```

```
            continue
        behind = tr.s < so
        if behind.all() and (np.abs(tr.l - o.l) < CAR_W + 0.5).any():
            s_hi = np.minimum(s_hi, so - CAR_L - 3.0)                # stays behind: bound s
        elif close.any():                                             # passes beside: bound l by side
            side = np.sign(np.mean(tr.l[close]) - o.l)
            if side > 0:  l_lo[close] = np.maximum(l_lo[close], o.l + CAR_W + 0.35)
            else:         l_hi[close] = np.minimum(l_hi[close], o.l - CAR_W - 0.35)

    def solve(x0vec, lo, hi, fixed=3):
        lo = lo.copy(); hi = hi.copy()
        lo[:fixed] = x0vec[:fixed] - 1e-6; hi[:fixed] = x0vec[:fixed] + 1e-6
        lo = np.minimum(lo, x0vec - 1e-6 + 0) if False else lo
        hi = np.maximum(hi, lo + 1e-6)
        A = np.vstack([np.sqrt(w_jerk) * D3, np.sqrt(w_dev) * np.eye(n)])
        b = np.concatenate([np.zeros(n - 3), np.sqrt(w_dev) * x0vec])
        x0c = np.clip(x0vec, lo, hi)
        return lsq_linear(A, b, bounds=(lo, hi), method="bvls", lsmr_tol="auto").x

    l_new = solve(tr.l, l_lo, l_hi)
    s_new = solve(tr.s, np.full(n, -np.inf), s_hi)
    s_new = np.maximum.accumulate(s_new)
    sd = np.gradient(s_new, dt); sdd = np.gradient(sd, dt)
    ld = np.gradient(l_new, dt); ldd = np.gradient(ld, dt)
    return Traj(t, s_new, l_new, sd, ld, sdd, ldd, tr.cost, tr.kind + "|opt")


def check_limits(tr: Traj):
    """Dynamic feasibility report for a trajectory."""
    return dict(max_acc=float(tr.sdd.max()), max_dec=float(tr.sdd.min()), max_lat_acc=float(np.abs(tr.ldd).max()),
                v_max=float(tr.sd.max()), jerk_rms=tr.jerk_rms(),
                ok=bool(tr.sdd.max() <= A_MAX + 0.3 and tr.sdd.min() >= A_MIN - 0.3 and np.abs(tr.ldd).max() <= 3.0 and tr.sd.min()
     >= -0.2))
```

Listing 6: control.py

```
"""
Vehicle model, reference path and trajectory-tracking controllers.
  Pure pursuit, Stanley, and a time-varying linear MPC (condensed bounded least squares) with online
  adaptation of the steering-effectiveness gain (NLMS on gyro yaw rate).
"""
import numpy as np
from scipy.optimize import lsq_linear

WHEELBASE, DELTA_MAX, DELTA_RATE = 2.7, 0.5, 0.7


def wrap(a):
    return (a + np.pi) % (2 * np.pi) - np.pi


class RefPath:
    """Smooth S-curve reference road, dense (ds = 0.1 m); Frenet <-> Cartesian conversion."""
    def __init__(self, length=700.0, ds=0.1):
        self.ds = ds
        self.s = np.arange(0, length, ds)
        self.th = 0.30 * np.sin(2 * np.pi * self.s / 260.0)
        self.x = np.cumsum(np.cos(self.th)) * ds
        self.y = np.cumsum(np.sin(self.th)) * ds
        self.kappa = np.gradient(self.th, ds)

    def at(self, s):
        s = np.clip(s, 0, self.s[-1] - 1e-3)
        return (np.interp(s, self.s, self.x), np.interp(s, self.s, self.y),
                np.interp(s, self.s, self.th), np.interp(s, self.s, self.kappa))

    def to_cart(self, s, l):
        x, y, th, _ = self.at(s)
        return x - l * np.sin(th), y + l * np.cos(th)

    def project(self, x, y, hint=None):
        i0, i1 = (0, len(self.s)) if hint is None else (max(int((hint - 20) / self.ds), 0), min(int((hint + 40) / self.ds), len(self.
     s)))
        d2 = (self.x[i0:i1] - x) ** 2 + (self.y[i0:i1] - y) ** 2
        i = i0 + int(np.argmin(d2))
        th = self.th[i]
        l = -np.sin(th) * (x - self.x[i]) + np.cos(th) * (y - self.y[i])
        s = self.s[i] + np.cos(th) * (x - self.x[i]) + np.sin(th) * (y - self.y[i])
        return s, l, i


class Vehicle:
    """Kinematic bicycle + first-order steering/acceleration actuators, unknown steering gain, latency."""
    def __init__(self, x, y, th, v, k_true=1.0, tau_steer=0.15, tau_acc=0.25, latency=0.0, dt=0.05):
```

```
        self.x, self.y, self.th, self.v = x, y, th, v
        self.delta, self.acc = 0.0, 0.0
        self.k_true, self.tau_s, self.tau_a, self.dt = k_true, tau_steer, tau_acc, dt
        self.buf = [(0.0, 0.0)] * int(round(latency / dt))

    def step(self, delta_cmd, acc_cmd):
        self.buf.append((delta_cmd, acc_cmd)); d_c, a_c = self.buf.pop(0)
        d_c = np.clip(self.k_true * d_c, -DELTA_MAX, DELTA_MAX)
        dd = np.clip((d_c - self.delta) / self.tau_s, -DELTA_RATE, DELTA_RATE)
        self.delta += dd * self.dt
        self.acc += (np.clip(a_c, -6, 3) - self.acc) / self.tau_a * self.dt
        self.v = max(0.0, self.v + self.acc * self.dt)
        self.x += self.v * np.cos(self.th) * self.dt
        self.y += self.v * np.sin(self.th) * self.dt
        self.th = wrap(self.th + self.v / WHEELBASE * np.tan(self.delta) * self.dt)
        return self.v / WHEELBASE * np.tan(self.delta)          # true yaw rate (gyro, before noise)


def tracking_errors(x, y, th, xr, yr, thr):
    ey = -np.sin(thr) * (x - xr) + np.cos(thr) * (y - yr)
    return ey, wrap(th - thr)


class PurePursuit:
    name = "Pure pursuit"
    def command(self, pose, v, ref, **_):
        x, y, th = pose
        Ld = max(4.0, 0.9 * v)
        px, py = ref["xs"], ref["ys"]
        d = np.hypot(px - x, py - y)
        j = int(np.argmax(d >= Ld)) if (d >= Ld).any() else len(d) - 1
        alpha = wrap(np.arctan2(py[j] - y, px[j] - x) - th)
        return float(np.clip(np.arctan2(2 * WHEELBASE * np.sin(alpha), Ld), -DELTA_MAX, DELTA_MAX))


class Stanley:
    name = "Stanley"
    def __init__(self, k=1.5):
        self.k = k
    def command(self, pose, v, ref, **_):
        x, y, th = pose
        ey, eth = tracking_errors(x, y, th, ref["x"], ref["y"], ref["th"])
        ff = np.arctan(WHEELBASE * ref["kappa"])
        return float(np.clip(ff - eth - np.arctan2(self.k * ey, v + 1.0), -DELTA_MAX, DELTA_MAX))


class AdaptiveMPC:
    """
    Error model (about the reference):  e_y' = v e_th,  e_th' = (k v / L) u,  u = delta - delta_ff.
    Time-varying in v (speed profile of the reference), k = steering gain estimated online (NLMS).
    Condensed QP:  min sum q_y e_y^2 + q_th e_th^2 + r u^2 + r_d du^2   s.t. |delta_ff + u| <= delta_max.
    """
    def __init__(self, N=15, dt=0.1, adapt=True, qy=1.0, qth=2.0, r=1.0, rd=20.0, mu=0.05):
        self.N, self.dt, self.adapt, self.mu = N, dt, adapt, mu
        self.q, self.r, self.rd = (qy, qth), r, rd
        self.k_hat, self.u_prev, self.delta_filt = 1.0, 0.0, 0.0
        self.name = "Adaptive MPC" if adapt else "MPC (fixed model)"

    def estimate(self, v, yaw_rate_meas, delta_cmd, dt):
        """NLMS update of the steering gain from gyro yaw rate (regressor = commanded steer through the known lag)."""
        self.delta_filt += (delta_cmd - self.delta_filt) / 0.15 * dt
        phi = v / WHEELBASE * np.tan(self.delta_filt)
        if self.adapt and abs(phi) > 0.01 and v > 2.0:
            self.k_hat += self.mu * phi * (yaw_rate_meas - self.k_hat * phi) / (1e-3 + phi ** 2)
            self.k_hat = float(np.clip(self.k_hat, 0.4, 1.6))

    def command(self, pose, v, ref, **_):
        N, dt = self.N, self.dt
        x, y, th = pose
        ey, eth = tracking_errors(x, y, th, ref["x"], ref["y"], ref["th"])
        vs, kap = ref["vN"], ref["kN"]                    # reference speed / curvature along horizon (N,)
        ff = np.arctan(WHEELBASE * kap) / self.k_hat       # feed-forward steer needed with estimated gain
        # condensed prediction  X = Phi x0 + Gam u   (X stacks x_1..x_N)
        Phi = np.zeros((2 * N, 2)); Gam = np.zeros((2 * N, N))
        P_prev, G_prev = np.eye(2), np.zeros((2, N))
        for k in range(N):
            A = np.array([[1.0, dt * vs[k]], [0.0, 1.0]])
            B = np.array([0.0, dt * self.k_hat * vs[k] / WHEELBASE])
            P_prev = A @ P_prev
            G_prev = A @ G_prev
            G_prev[:, k] = B
            Phi[2 * k:2 * k + 2] = P_prev
            Gam[2 * k:2 * k + 2] = G_prev
        x0 = np.array([ey, eth])
        sq = np.sqrt(np.tile(self.q, N))
```

```
        rows = [sq[:, None] * Gam, np.sqrt(self.r) * np.eye(N)]
        rhs = [-(sq * (Phi @ x0)), np.zeros(N)]
        D = np.eye(N) - np.eye(N, k=-1)
        rows.append(np.sqrt(self.rd) * D); e0 = np.zeros(N); e0[0] = np.sqrt(self.rd) * self.u_prev
        rhs.append(e0)
        lo, hi = -DELTA_MAX - ff, DELTA_MAX - ff
        sol = lsq_linear(np.vstack(rows), np.concatenate(rhs), bounds=(lo, hi), method="bvls")
        u = sol.x[0]
        self.u_prev = float(u)
        return float(np.clip(ff[0] + u, -DELTA_MAX, DELTA_MAX))


class SpeedController:
    """Feed-forward acceleration + PI on speed + along-track position error."""
    def __init__(self, kp=1.2, ki=0.15, ks=0.25):
        self.kp, self.ki, self.ks, self.I = kp, ki, ks, 0.0
    def command(self, v, v_ref, a_ref, s_err, dt):
        e = v_ref - v
        self.I = float(np.clip(self.I + e * dt, -3, 3))
        return float(np.clip(a_ref + self.kp * e + self.ki * self.I + self.ks * s_err, -6, 3))
```

Listing 7: closed_loop.py

```
"""
Closed-loop evaluation: lattice planner -> QP trajectory optimisation -> tracking controller -> vehicle.
Run: python closed_loop.py [--plot]   (writes results/closed_loop.json and figures/closed_loop.png)
"""
import sys, json, time, os
import numpy as np
from planning import LatticePlanner, FState, Obstacle, optimise_trajectory, check_limits, collides, CAR_L, CAR_W
from control import (RefPath, Vehicle, PurePursuit, Stanley, AdaptiveMPC, SpeedController, wrap, WHEELBASE)

DT, REPLAN = 0.05, 0.5


def make_scenario():
    return [Obstacle(45.0, 0.0, 5.0), Obstacle(180.0, 0.0, 0.0)]     # slow lead + parked car in the ego lane


def traj_to_cart(tr, ref, dt=0.05):
    """Resample a Frenet trajectory on a dt grid and convert to Cartesian reference signals."""
    t = np.arange(0, tr.t[-1] + 1e-9, dt)
    s = np.interp(t, tr.t, tr.s); l = np.interp(t, tr.t, tr.l)
    sd = np.interp(t, tr.t, tr.sd); sdd = np.interp(t, tr.t, tr.sdd)
    x, y = ref.to_cart(s, l)
    dx, dy = np.gradient(x, dt), np.gradient(y, dt)
    th = np.unwrap(np.arctan2(dy, dx)); v = np.hypot(dx, dy)
    kappa = np.gradient(th, dt) / np.maximum(v, 0.5)
    return dict(t=t, s=s, l=l, x=x, y=y, th=th, v=v, kappa=kappa, sd=sd, sdd=sdd)


def run(controller, k_true=1.0, latency=0.0, pos_noise=0.0, yaw_noise=0.0, seed=0, v0=8.0, t_end=60.0,
        use_opt=True, record=False):
    rng = np.random.default_rng(seed)
    ref = RefPath(); planner = LatticePlanner(v_des=12.0)
    obs = make_scenario()
    x0, y0, th0, _ = ref.at(0.0)
    veh = Vehicle(x0, y0, th0, v0, k_true=k_true, latency=latency, dt=DT)
    sp = SpeedController(); ctrl = controller
    if hasattr(ctrl, "k_hat"): ctrl.k_hat, ctrl.u_prev, ctrl.delta_filt = 1.0, 0.0, 0.0
    t, s_hint = 0.0, 0.0
    traj = None; t_plan = -1e9; cart = None
    st = FState(0.0, v0, 0.0, 0.0, 0.0, 0.0)
    ey_log, es_log, clr_log, lat_acc, plan_ms, cand_n, log = [], [], [], [], [], [], []
    collisions, infeasible, replans = 0, 0, 0
    yaw_rate = 0.0; delta_cmd = 0.0
    while t < t_end:
        s_true, l_true, ihint = ref.project(veh.x, veh.y, s_hint); s_hint = s_true
        if s_true > 330.0:
            break
        # ---- measurement with injected localisation error
        mx = veh.x + rng.normal(0, pos_noise); my = veh.y + rng.normal(0, pos_noise)
        mth = veh.th + rng.normal(0, yaw_noise)
        # ---- replan
        if traj is None or t - t_plan >= REPLAN - 1e-9:
            tau = t - t_plan
            if traj is not None:
                k = min(int(round(tau / planner.dt)), len(traj.t) - 1)
                st = FState(traj.s[k], traj.sd[k], traj.sdd[k], traj.l[k], traj.ld[k], traj.ldd[k])
                s_m, l_m, _ = ref.project(mx, my, s_hint)
                if abs(l_m - st.l) > 1.0 or abs(s_m - st.s) > 4.0:        # tracking lost: restart from measurement
                    st = FState(s_m, veh.v, 0.0, l_m, 0.0, 0.0)
            else:
                st = FState(s_true, veh.v, 0.0, l_true, 0.0, 0.0)
```

```
            t0 = time.perf_counter()
            tr, n = planner.plan(st, [Obstacle(o.s, o.l, o.v) for o in obs])
            if use_opt:
                op = optimise_trajectory(tr, obs)
                if not collides(op.t, op.s, op.l, op.sd, obs) and check_limits(op)["ok"]:
                    tr = op
                else:
                    infeasible += 1
            plan_ms.append((time.perf_counter() - t0) * 1e3); cand_n.append(n)
            traj, t_plan = tr, t
            # shift so that traj.t starts at plan time; obstacles in the plan already start at plan time
            cart = traj_to_cart(traj, ref); replans += 1
            plan_obs_t = t
        # ---- reference at current time
        tau = min(t - t_plan, cart["t"][-1] - 2.0)
        i = int(round(tau / DT))
        r = {k: cart[k][i] for k in ("x", "y", "th", "kappa", "v", "s", "sd", "sdd")}
        idx = np.minimum(i + np.round(0.1 / DT * np.arange(1, 16)).astype(int), len(cart["t"]) - 1)
        r["vN"], r["kN"] = cart["v"][idx], cart["kappa"][idx]
        j = np.minimum(i + np.arange(0, 120, 6), len(cart["t"]) - 1)
        r["xs"], r["ys"] = cart["x"][j], cart["y"][j]
        # ---- control
        delta_cmd = ctrl.command((mx, my, mth), veh.v, r)
        s_m, _, _ = ref.project(mx, my, s_hint)
        a_cmd = sp.command(veh.v, r["sd"] * 1.0 if False else r["v"], r["sdd"], r["s"] - s_m, DT)
        yaw_rate = veh.step(delta_cmd, a_cmd)
        if hasattr(ctrl, "estimate"):
            ctrl.estimate(veh.v, yaw_rate + rng.normal(0, 0.005), delta_cmd, DT)
        # ---- world update + metrics (ground truth)
        for o in obs:
            o.s += o.v * DT
        t += DT
        s_tr, l_tr, _ = ref.project(veh.x, veh.y, s_hint)
        ey, _ = (lambda e: e)((-np.sin(r["th"]) * (veh.x - r["x"]) + np.cos(r["th"]) * (veh.y - r["y"]), 0))
        ey_log.append(ey); es_log.append(s_tr - r["s"]); lat_acc.append(abs(veh.v * yaw_rate))
        c = 1e9
        for o in obs:
            gs, gl = abs(s_tr - o.s) - CAR_L, abs(l_tr - o.l) - CAR_W
            c = min(c, max(gs, gl))
            if gs < 0 and gl < 0:
                collisions += 1
        clr_log.append(c)
        if record:
            log.append((t, veh.x, veh.y, veh.v, s_tr, l_tr, delta_cmd, ey, getattr(ctrl, "k_hat", 1.0)))
    ey = np.array(ey_log)
    out = dict(controller=ctrl.name, completed=bool(s_hint > 325.0), time=round(t, 1),
               rms_ey=float(np.sqrt(np.mean(ey ** 2))), max_ey=float(np.max(np.abs(ey))),
               collision_steps=int(collisions), min_clearance=float(np.min(clr_log)),
               max_lat_acc=float(np.max(lat_acc)), plan_ms_mean=float(np.mean(plan_ms)),
               plan_ms_p95=float(np.percentile(plan_ms, 95)), replans=replans, opt_rejected=infeasible,
               k_hat_final=float(getattr(ctrl, "k_hat", 1.0)))
    return (out, np.array(log), obs) if record else out


CONDITIONS = {
    "nominal": dict(),
    "gain_0.7+latency_0.1s": dict(k_true=0.7, latency=0.10),
    "loc_noise_0.3m": dict(pos_noise=0.3, yaw_noise=0.01),
    "all_faults": dict(k_true=0.7, latency=0.10, pos_noise=0.3, yaw_noise=0.01),
}


def controllers():
    return [PurePursuit(), Stanley(), AdaptiveMPC(adapt=False), AdaptiveMPC(adapt=True)]


if __name__ == "__main__":
    os.makedirs("results", exist_ok=True); os.makedirs("figures", exist_ok=True)
    results = {}
    print(f"{'condition':24s} {'controller':20s} {'done':>5s} {'RMS ey':>7s} {'max ey':>7s} {'coll':>5s} {'clr[m]':>7s} {'plan ms':>8
      s}")
    for cname, kw in CONDITIONS.items():
        results[cname] = []
        for c in controllers():
            r = run(c, **kw); results[cname].append(r)
            print(f"{cname:24s} {r['controller']:20s} {str(r['completed']):>5s} {r['rms_ey']:7.3f} {r['max_ey']:7.2f} "
                  f"{r['collision_steps']:5d} {r['min_clearance']:7.2f} {r['plan_ms_mean']:8.1f}")
    json.dump(results, open("results/closed_loop.json", "w"), indent=1)
    if "--plot" in sys.argv:
        import matplotlib; matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        ref = RefPath()
        fig, ax = plt.subplots(1, 3, figsize=(15, 4))
        for c in controllers():
            r, log, obs = run(c, record=True, **CONDITIONS["all_faults"])
```

```
            ax[0].plot(log[:, 4], log[:, 5], label=c.name)
            ax[1].plot(log[:, 0], log[:, 7], label=c.name)
            if c.name == "Adaptive MPC": ax[2].plot(log[:, 0], log[:, 8], "g")
        for o0 in make_scenario():
            ax[0].plot([o0.s, o0.s + 4.5], [o0.l, o0.l], "k", lw=6, alpha=.4)
        ax[0].set_xlabel("s [m]"); ax[0].set_ylabel("l [m]"); ax[0].set_title("Path in Frenet frame (all faults)"); ax[0].legend(fontsize=7)
        ax[1].set_xlabel("t [s]"); ax[1].set_ylabel("tracking error e_y [m]"); ax[1].set_title("Lateral tracking error"); ax[1].legend(fontsize=7)
        ax[2].axhline(0.7, ls="--", c="r"); ax[2].set_xlabel("t [s]"); ax[2].set_ylabel("estimated steering gain"); ax[2].set_title("Adaptive MPC gain estimate (true 0.7)")
        plt.tight_layout(); plt.savefig("figures/closed_loop.png", dpi=130); print("saved figures/closed_loop.png")
```

Listing 8: fault_injection.py

```
"""
Fault-injection campaigns.
  (A) Localisation: scan blackout, high outlier rate, gyro bias, high sensor noise, map change.
  (B) Decision making: perception faults (latency, dropout, speed bias, circulating-traffic timing error)
      fed to the behaviour tree, evaluated with and without a degraded-mode supervisor.
Run: python fault_injection.py   (writes results/fault_injection.json)
"""
import json, os, copy
import numpy as np
import behavior_tree as bt
import run_localization as rl
from localization import make_world

# ---------------------------------------------------------------- (A) localisation
def localisation_campaign(n_seeds=5):
    map_pts, path, heading, total = make_world(seed=0)
    # map change: delete every third 12 m building block on both sides (construction / demolished facade)
    s_map = np.cumsum(np.r_[0, np.linalg.norm(np.diff(path, axis=0), axis=1)])
    nearest = np.argmin(((map_pts[:, None, :] - path[None, ::4, :]) ** 2).sum(-1), axis=1) * 4
    blk = (s_map[nearest] // 12.0).astype(int)
    changed = map_pts[blk % 3 != 0]
    changed2 = map_pts[blk % 3 == 0]
    faults = {
        "baseline": {},
        "blackout_5s": dict(blackout=(20.0, 25.0)),
        "outliers_40pct": dict(outlier_frac=0.40),
        "gyro_bias_0.03": dict(gyro_bias=0.03),
        "sensor_noise_0.3m": dict(sigma=0.30),
        "map_change_33pct": dict(world_pts=changed),
        "blackout_20s": dict(blackout=(15.0, 35.0)),
        "outliers_70pct": dict(outlier_frac=0.70),
        "map_change_67pct": dict(world_pts=changed2),
        "severe_combo": dict(blackout=(15.0, 30.0), outlier_frac=0.5, gyro_bias=0.05, world_pts=changed),
    }
    res = {}
    for name, f in faults.items():
        rows = []
        for seed in range(n_seeds):
            truth, est = rl.drive(seed, map_pts, path, heading, methods=("dr", "ndt", "pf", "ndtpf"), faults=f)
            for m in ("ndt", "ndtpf"):
                pos, _ = rl.errors(truth, est[m])
                rows.append((m, np.sqrt((pos ** 2).mean()), pos.max(), float(pos[-50:].mean() < 1.0)))
            # divergence detector quality: score<0.25 for >=5 consecutive steps vs true error>2 m
            pos, _ = rl.errors(truth, est["ndtpf"]); low = np.r_[est["scores"][:1], est["scores"]] < 0.25
            run = np.convolve(low, np.ones(5), "same") >= 5
            bad = pos > 2.0
            rows.append(("detect", float((run & bad).sum()), float(bad.sum()), float((run & ~bad).sum()), float((~bad).sum())))
        out = {}
        for m in ("ndt", "ndtpf"):
            r = np.array([x[1:] for x in rows if x[0] == m]); out[m] = dict(rmse=float(r[:, 0].mean()), max=float(r[:, 1].mean()), converged=float(r[:, 2].mean()))
        d = np.array([x[1:5] for x in rows if x[0] == "detect"]).sum(0)
        out["detector"] = dict(steps_err_gt_2m=int(d[1]), recall_when_err_gt_2m=(float(d[0] / d[1]) if d[1] else None),
                               false_alarm_rate=float(d[2] / d[3]))
        res[name] = out
        print(f"{name:20s} NDT rmse {out['ndt']['rmse']:6.2f} max {out['ndt']['max']:6.2f} conv {out['ndt']['converged']*100:3.0f}% | "
              f"NDT-PF rmse {out['ndtpf']['rmse']:6.2f} max {out['ndtpf']['max']:6.2f} conv {out['ndtpf']['converged']*100:3.0f}% | "
              f"det recall {('n/a' if out['detector']['recall_when_err_gt_2m'] is None else format(out['detector']['recall_when_err_gt_2m'], '.2f'))} FA {out['detector']['false_alarm_rate']:.3f}")
    return res


# ---------------------------------------------------------------- (B) decision making
class Perceived:
    """What the behaviour tree believes. Built from the true world + a fault model; true world is used for physics."""
    def __init__(self, w, fault, hist, rng):
        for k, v in w.__dict__.items():
            setattr(self, k, copy.copy(v))
```

```
        self.degraded_flag = False
        kind = fault.get("kind")
        if kind == "latency":                                   # stale object list (positions from `lag` seconds ago)
            lag = int(fault["lag"] / 0.1); old = hist[max(0, len(hist) - 1 - lag)]
            self.lead_s, self.onc_s = old["lead_s"], old["onc_s"]
            self.degraded_flag = True
        elif kind == "dropout":                                 # oncoming car not detected during a window
            t0, t1 = fault["window"]
            if t0 <= w.t < t1:
                self.onc_s = 1e6; self.degraded_flag = True
        elif kind == "speed_bias":                              # oncoming speed under-estimated
            self.onc_v = w.onc_v * fault["factor"]
        elif kind == "cv_timing":                               # circulating traffic timing error (s)
            self.cv_times = w.cv_times + rng.normal(0, fault["sigma"], len(w.cv_times))


def run_faulted(seed, fault, supervisor):
    rng = np.random.default_rng(10_000 + seed)
    w, tree = bt.World(seed), bt.build_tree(); hist = []
    while w.s < bt.ROUTE_END and w.t < 150:
        hist.append(dict(lead_s=w.lead_s, onc_s=w.onc_s))
        pw = Perceived(w, fault, hist, rng)
        bb = bt.bb_from_world(pw); bb["cmd"] = dict(speed=0.0, lane=0)
        bb["degraded"] = supervisor and pw.degraded_flag
        tree.tick(bb)
        # commands act on the TRUE world; overtake bookkeeping (counters) lives on the true world
        w.step(bb["cmd"])
    return dict(done=w.s >= bt.ROUTE_END, coll=w.collisions > 0)


def decision_campaign(n=200):
    faults = {
        "none": dict(),
        "latency_0.5s": dict(kind="latency", lag=0.5),
        "latency_1.0s": dict(kind="latency", lag=1.0),
        "oncoming_dropout_3s": dict(kind="dropout", window=(8.0, 11.0)),
        "oncoming_speed_x0.6": dict(kind="speed_bias", factor=0.6),
        "cv_timing_sigma_1.5s": dict(kind="cv_timing", sigma=1.5),
    }
    res = {}
    for name, f in faults.items():
        res[name] = {}
        for sup in (False, True):
            r = [run_faulted(s, f, sup) for s in range(n)]
            res[name]["supervisor" if sup else "no_supervisor"] = dict(
                collision_rate=float(np.mean([x["coll"] for x in r])), completion=float(np.mean([x["done"] for x in r])))
        a, b = res[name]["no_supervisor"], res[name]["supervisor"]
        print(f"{name:24s} collisions {a['collision_rate']*100:5.1f}% -> {b['collision_rate']*100:5.1f}% (with supervisor) |
  completion {a['completion']*100:5.1f}% / {b['completion']*100:5.1f}%")
    return res


if __name__ == "__main__":
    os.makedirs("results", exist_ok=True)
    print("== (A) localisation faults, 5 seeds ==")
    A = localisation_campaign()
    print("\n== (B) decision-making faults, 200 episodes each ==")
    B = decision_campaign()
    json.dump(dict(localisation=A, decision=B), open("results/fault_injection.json", "w"), indent=1)
```

Listing 9: datasets.py

```
"""
Public-dataset utilities: KITTI odometry loader (velodyne .bin, poses, calib) and trajectory metrics
(ATE with SE(2)/SE(3) alignment, RPE). NOTE: the loader is validated here on synthetic data written in the
KITTI file format (tests/test_datasets.py); it has NOT yet been run on the real KITTI dataset.

Expected layout:  <root>/sequences/00/velodyne/000000.bin ... ,  <root>/poses/00.txt
"""
import os
import numpy as np


def load_velodyne(path):
    return np.fromfile(path, dtype=np.float32).reshape(-1, 4)          # x, y, z, intensity


def load_poses(path):
    """KITTI poses: each line = 12 numbers of a 3x4 [R|t] camera-to-world matrix. Returns (N,4,4)."""
    P = np.loadtxt(path).reshape(-1, 3, 4)
    T = np.tile(np.eye(4), (len(P), 1, 1)); T[:, :3, :4] = P
    return T


```

```python
def scan_to_2d(points, z_min=-1.5, z_max=1.5, rng_max=40.0, n=120, rng=None):
    """Project a 3-D lidar scan to the 2-D point set used by the NDT-PF (height slab, range gate, subsample)."""
    rng = rng or np.random.default_rng(0)
    m = (points[:, 2] > z_min) & (points[:, 2] < z_max) & (np.hypot(points[:, 0], points[:, 1]) < rng_max)
    p = points[m, :2]
    return p[rng.choice(len(p), min(n, len(p)), replace=False)] if len(p) else p


def ate_rmse(est_xy, gt_xy, align=True):
    """Absolute trajectory error (RMSE, m) after optional rigid alignment (Umeyama without scale)."""
    est, gt = np.asarray(est_xy, float), np.asarray(gt_xy, float)
    if align:
        mu_e, mu_g = est.mean(0), gt.mean(0)
        H = (est - mu_e).T @ (gt - mu_g)
        U, _, Vt = np.linalg.svd(H)
        D = np.diag([1, np.sign(np.linalg.det(Vt.T @ U.T))])
        R = Vt.T @ D @ U.T
        est = (R @ (est - mu_e).T).T + mu_g
    return float(np.sqrt(np.mean(np.sum((est - gt) ** 2, axis=1))))


def rpe_translation(est_xy, gt_xy, delta=10):
    """Relative pose error: translation drift over `delta` frames (mean, m)."""
    de = est_xy[delta:] - est_xy[:-delta]; dg = gt_xy[delta:] - gt_xy[:-delta]
    return float(np.mean(np.linalg.norm(de - dg, axis=1)))


def kitti_sequence(root, seq="00"):
    vdir = os.path.join(root, "sequences", seq, "velodyne")
    files = sorted(f for f in os.listdir(vdir) if f.endswith(".bin"))
    poses = load_poses(os.path.join(root, "poses", f"{seq}.txt"))
    return [os.path.join(vdir, f) for f in files], poses
```

Listing 10: deploy/embedded_bench.py

```python
"""
Per-module latency benchmark (host CPU). Used to size the real-time budget before porting to an embedded
target: run the same script ON the target to get target numbers (none are reported for hardware not tested).
Run: python deploy/embedded_bench.py  -> results/latency.json
"""
import os, sys, json, time, platform
import numpy as np
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from localization import make_world, simulate_scan, NDTMap, NDTParticleFilter, ndt_align
from planning import LatticePlanner, FState, Obstacle, optimise_trajectory
from control import AdaptiveMPC, RefPath
import behavior_tree as bt


def timeit(fn, n=60, warm=5):
    for _ in range(warm): fn()
    ts = []
    for _ in range(n):
        t0 = time.perf_counter(); fn(); ts.append((time.perf_counter() - t0) * 1e3)
    ts = np.array(ts)
    return dict(p50_ms=float(np.percentile(ts, 50)), p95_ms=float(np.percentile(ts, 95)), max_ms=float(ts.max()))


def main():
    rng = np.random.default_rng(0)
    pts, path, heading, _ = make_world(0)
    ndt = NDTMap(pts); pose = np.array([*path[200], heading[200]]); scan = simulate_scan(pose, pts, rng)
    res = {"host": platform.processor() or platform.machine(), "python": platform.python_version(),
           "numpy": np.__version__, "cpu_count": os.cpu_count()}
    for n in (100, 300, 1000):
        pf = NDTParticleFilter(ndt, pose, n=n, use_injection=False)
        res[f"pf_update_{n}_particles"] = timeit(lambda: (pf.predict(5.0, 0.0, 0.1), pf.update(scan)))
    res["ndt_align_6_iters"] = timeit(lambda: ndt_align(ndt, scan, pose + [0.5, 0.3, 0.03], iters=6))
    pf = NDTParticleFilter(ndt, pose, n=300, use_injection=True)
    res["ndt_pf_full_cycle_300"] = timeit(lambda: (pf.predict(5.0, 0.0, 0.1), pf.update(scan)))
    pl = LatticePlanner(); obs = [Obstacle(60, 0, 5.0), Obstacle(140, 3.5, 0.0)]; st = FState(0, 10, 0, 0, 0, 0)
    res["lattice_plan"] = timeit(lambda: pl.plan(st, obs), n=40)
    tr, _ = pl.plan(st, obs)
    res["qp_trajectory_opt"] = timeit(lambda: optimise_trajectory(tr, obs), n=40)
    ref = RefPath(); mpc = AdaptiveMPC()
    r = dict(x=0.0, y=0.5, th=0.0, kappa=0.0, vN=np.full(15, 10.0), kN=np.zeros(15))
    res["mpc_step"] = timeit(lambda: mpc.command((0.0, 0.5, 0.0), 10.0, r), n=200)
    w = bt.World(0); tree = bt.build_tree()
    def tick():
        bb = bt.bb_from_world(w); bb["cmd"] = dict(speed=0.0, lane=0); tree.tick(bb)
    res["bt_tick"] = timeit(tick, n=500)
    budget = {"localisation_10Hz": ("ndt_pf_full_cycle_300", 100.0), "planning_2Hz": ("lattice_plan", 500.0),
              "control_20Hz": ("mpc_step", 50.0), "decision_10Hz": ("bt_tick", 100.0)}
    res["budget_check"] = {k: dict(module=m, p95_ms=res[m]["p95_ms"], budget_ms=b, ok=res[m]["p95_ms"] < b) for k, (m, b) in budget.
     items()}
```

```
    os.makedirs("results", exist_ok=True); json.dump(res, open("results/latency.json", "w"), indent=1)
    for k, v in res.items():
        if isinstance(v, dict) and "p50_ms" in v:
            print(f"{k:28s} p50 {v['p50_ms']:8.2f} ms   p95 {v['p95_ms']:8.2f} ms")
    for k, v in res["budget_check"].items():
        print(f"budget {k:20s} p95 {v['p95_ms']:7.2f} / {v['budget_ms']:.0f} ms  {'OK' if v['ok'] else 'EXCEEDED'}")


if __name__ == "__main__":
    main()
```

Listing 11: carla_bt_runner.py (not yet executed)

```
"""
Runs the behaviour tree (behavior_tree.py, unchanged) inside CARLA 0.9.x.

STATUS: written against the CARLA Python API but NOT yet executed in this project (no CARLA server was
available during development). Treat as an integration skeleton to be validated; see the report, Sec. 'Planned'.

The BT only reads attributes of a world object (s, v, lat, t, lead_s, lead_v, onc_s, onc_v, cv_times).
CarlaAdapter fills those from CARLA actors; the BT output (target speed, target lane) is converted to
carla.VehicleControl with a PI speed loop and pure-pursuit steering from control.py.

Usage:  python carla_bt_runner.py --host localhost --port 2000 --town Town03 --seed 0
"""
import argparse, math, time
import numpy as np
import behavior_tree as bt
from control import wrap, WHEELBASE

try:
    import carla
except ImportError:                       # allows importing this file (and unit-testing the adapter) without CARLA
    carla = None


class CarlaAdapter:
    """Maps CARLA actor states to the scalar world model used by the behaviour tree."""
    def __init__(self, ego, lead, oncoming, route, cv_times=()):
        self.ego, self.lead, self.onc, self.route = ego, lead, oncoming, route   # route: (N,2) polyline, left-hand lane = +l
        self.cum = np.r_[0, np.cumsum(np.linalg.norm(np.diff(route, axis=0), axis=1))]
        self.cv_times = np.array(cv_times, float)
        self.t0 = time.time(); self.lat = 0.0
        self.t = self.s = self.v = self.lead_s = self.lead_v = self.onc_s = self.onc_v = 0.0
        self.collisions = 0; self.overtakes = 0

    def station(self, loc):
        d = np.hypot(self.route[:, 0] - loc.x, self.route[:, 1] - loc.y); i = int(np.argmin(d))
        return float(self.cum[i]), i

    def update(self):
        self.t = time.time() - self.t0
        for name, actor in (("ego", self.ego), ("lead", self.lead), ("onc", self.onc)):
            s, i = self.station(actor.get_location())
            vel = actor.get_velocity(); fwd = actor.get_transform().get_forward_vector()
            v = vel.x * fwd.x + vel.y * fwd.y
            if name == "ego":
                self.s, self.v = s, v
                tang = self.route[min(i + 1, len(self.route) - 1)] - self.route[i]
                nrm = np.array([-tang[1], tang[0]]) / (np.linalg.norm(tang) + 1e-9)
                loc = actor.get_location()
                self.lat = float(np.clip(np.dot([loc.x - self.route[i, 0], loc.y - self.route[i, 1]], nrm) / 3.5, 0, 1))
            elif name == "lead":
                self.lead_s, self.lead_v = s, v
            else:
                self.onc_s, self.onc_v = s, -abs(v)           # oncoming: station decreases with time

    def command_to_control(self, cmd, steer):
        ctrl = carla.VehicleControl()
        err = cmd["speed"] - self.v
        ctrl.throttle = float(np.clip(0.4 * err, 0, 1)); ctrl.brake = float(np.clip(-0.3 * err, 0, 1))
        ctrl.steer = float(np.clip(steer / 0.5, -1, 1))
        return ctrl


def pure_pursuit_steer(ego_tf, target_xy):
    yaw = math.radians(ego_tf.rotation.yaw); loc = ego_tf.location
    alpha = wrap(math.atan2(target_xy[1] - loc.y, target_xy[0] - loc.x) - yaw)
    Ld = max(4.0, math.hypot(target_xy[0] - loc.x, target_xy[1] - loc.y))
    return math.atan2(2 * WHEELBASE * math.sin(alpha), Ld)


def main():
    if carla is None:
```

```
        raise SystemExit("carla package not installed (pip install carla==0.9.15 matching your server)")
    ap = argparse.ArgumentParser()
    ap.add_argument("--host", default="localhost"); ap.add_argument("--port", type=int, default=2000)
    ap.add_argument("--town", default="Town04"); ap.add_argument("--seed", type=int, default=0)
    ap.add_argument("--duration", type=float, default=90.0)
    a = ap.parse_args()
    rng = np.random.default_rng(a.seed)
    client = carla.Client(a.host, a.port); client.set_timeout(10.0)
    world = client.load_world(a.town); m = world.get_map()
    settings = world.get_settings(); settings.synchronous_mode = True; settings.fixed_delta_seconds = 0.05
    world.apply_settings(settings)
    bp = world.get_blueprint_library()
    start = m.get_spawn_points()[0]
    wp = m.get_waypoint(start.location)
    # reference polyline = centre of the ego lane for 420 m, left lane offset handled via lat in [0, 1]
    route, w = [], wp
    for _ in range(420):
        route.append([w.transform.location.x, w.transform.location.y]); w = w.next(1.0)[0]
    route = np.array(route)
    actors = []
    try:
        def spawn(model, tf):
            actor = world.spawn_actor(bp.find(model), tf); actors.append(actor); return actor
        ego = spawn("vehicle.tesla.model3", start)
        lead_tf = m.get_waypoint(start.location).next(rng.uniform(30, 60))[0].transform; lead_tf.location.z += 0.3
        lead = spawn("vehicle.audi.a2", lead_tf)
        lead.set_target_velocity(carla.Vector3D(0, 0, 0))
        onc_wp = (wp.get_left_lane() or wp).next(rng.uniform(180, 320))[0]
        onc_tf = onc_wp.transform; onc_tf.rotation.yaw += 180; onc_tf.location.z += 0.3
        onc = spawn("vehicle.audi.tt", onc_tf)
        adapter = CarlaAdapter(ego, lead, onc, route, cv_times=np.sort(rng.uniform(18, 60, 3)))
        tree = bt.build_tree(); lead_v = rng.uniform(3, 7); onc_v = rng.uniform(9, 13)
        while adapter.t < a.duration and adapter.s < bt.ROUTE_END:
            world.tick()
            # simple scripted traffic: constant speed along each actor's heading
            for actor, speed in ((lead, lead_v), (onc, onc_v)):
                f = actor.get_transform().get_forward_vector(); actor.set_target_velocity(carla.Vector3D(f.x * speed, f.y * speed, 0)
 )
            adapter.update()
            bb = bt.bb_from_world(adapter); bb["cmd"] = dict(speed=0.0, lane=0); tree.tick(bb)
            i = adapter.station(ego.get_location())[1]; tgt = route[min(i + 12, len(route) - 1)]
            ego.apply_control(adapter.command_to_control(bb["cmd"], pure_pursuit_steer(ego.get_transform(), tgt)))
            print(f"t={adapter.t:5.1f} s={adapter.s:6.1f} v={adapter.v:4.1f} lat={adapter.lat:.2f} leaf={bb['active'][-1] if bb['
 active'] else '-'}")
    finally:
        settings.synchronous_mode = False; world.apply_settings(settings)
        for actor in actors: actor.destroy()


if __name__ == "__main__":
    main()
```

Listing 12: deploy/ros2_node.py (not yet executed)

```
"""
ROS 2 (rclpy) wrapper: runs the behaviour tree at 10 Hz and publishes [target_speed, target_lane].
STATUS: not executed in this project (no ROS 2 installation was used); interface skeleton only.
Subscribes: /ego/odom, /lead/odom, /oncoming/odom (nav_msgs/Odometry, route-aligned x = station [m]).
Publishes : /bt/cmd (std_msgs/Float32MultiArray) -> [speed_mps, lane(0|1)]
"""
import os, sys
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import numpy as np
import behavior_tree as bt

try:
    import rclpy
    from rclpy.node import Node
    from nav_msgs.msg import Odometry
    from std_msgs.msg import Float32MultiArray
except ImportError:
    rclpy = None


class BTNode(Node if rclpy else object):
    def __init__(self):
        super().__init__("behavior_tree")
        self.tree = bt.build_tree(); self.w = bt.World(0)
        self.w.cv_times = np.array([])
        self.create_subscription(Odometry, "/ego/odom", self.on_ego, 10)
        self.create_subscription(Odometry, "/lead/odom", self.on_lead, 10)
        self.create_subscription(Odometry, "/oncoming/odom", self.on_onc, 10)
        self.pub = self.create_publisher(Float32MultiArray, "/bt/cmd", 10)
        self.create_timer(0.1, self.tick)
```

```
31
32      def on_ego(self, m):  self.w.s, self.w.v, self.w.lat = m.pose.pose.position.x, m.twist.twist.linear.x, float(np.clip(m.pose.pose.
          position.y / 3.5, 0, 1))
33      def on_lead(self, m): self.w.lead_s, self.w.lead_v = m.pose.pose.position.x, m.twist.twist.linear.x
34      def on_onc(self, m):  self.w.onc_s, self.w.onc_v = m.pose.pose.position.x, -abs(m.twist.twist.linear.x)
35
36      def tick(self):
37          self.w.t = self.get_clock().now().nanoseconds * 1e-9
38          bb = bt.bb_from_world(self.w); bb["cmd"] = dict(speed=0.0, lane=0); self.tree.tick(bb)
39          msg = Float32MultiArray(); msg.data = [float(bb["cmd"]["speed"]), float(bb["cmd"]["lane"])]; self.pub.publish(msg)
40
41
42  def main():
43      if rclpy is None:
44          raise SystemExit("rclpy not available: source your ROS 2 environment first")
45      rclpy.init(); rclpy.spin(BTNode()); rclpy.shutdown()
46
47
48  if __name__ == "__main__":
49      main()
```

Listing 13: tests/test_all.py

```
1   """Run: python -m tests.test_all   (fast regression tests for every module)"""
2   import os, sys
3   import numpy as np
4   sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
5   import behavior_tree as bt
6   from localization import make_world, simulate_scan, NDTMap, ndt_align
7   from planning import LatticePlanner, FState, Obstacle, optimise_trajectory, collides, check_limits
8   from control import RefPath, Vehicle, Stanley, AdaptiveMPC, tracking_errors
9
10
11  def test_bt_structure_and_safety():
12      tree = bt.build_tree()
13      assert tree.children[-1].name == "cruise"                       # root is total (never FAILURE)
14      for seed in range(40):
15          res, _ = bt.run_episode(seed)
16          assert res["done"] and res["collisions"] == 0, seed
17
18
19  def test_bt_never_overtakes_into_oncoming():
20      p = dict(lead_s=30, lead_v=4, onc_s=120, onc_v=12, cv_times=[])
21      res, log = bt.run_episode(0, params=p, trace=True)
22      assert res["collisions"] == 0 and res["done"]
23
24
25  def test_ndt_recovers_offset():
26      pts, path, hd, _ = make_world(0); rng = np.random.default_rng(1)
27      ndt = NDTMap(pts); pose = np.array([*path[300], hd[300]]); scan = simulate_scan(pose, pts, rng)
28      est, _ = ndt_align(ndt, scan, pose + [0.8, -0.6, 0.06], iters=10)
29      assert np.linalg.norm(est[:2] - pose[:2]) < 0.3
30
31
32  def test_planner_collision_free_and_feasible():
33      pl = LatticePlanner(); obs = [Obstacle(50, 0, 4.0), Obstacle(120, 3.5, 0.0)]
34      tr, n = pl.plan(FState(0, 10, 0, 0, 0, 0), obs)
35      assert n > 0 and not collides(tr.t, tr.s, tr.l, tr.sd, obs)
36      op = optimise_trajectory(tr, obs)
37      assert not collides(op.t, op.s, op.l, op.sd, obs) and check_limits(op)["ok"]
38      assert op.jerk_rms() <= tr.jerk_rms() + 1e-9                    # optimisation does not worsen jerk
39
40
41  def test_emergency_when_boxed_in():
42      pl = LatticePlanner(); obs = [Obstacle(15, 0, 0.0), Obstacle(15, 3.5, 0.0)]
43      tr, n = pl.plan(FState(0, 10, 0, 0, 0, 0), obs)
44      assert tr.kind == "emergency" and tr.sd[-1] < 0.5
45
46
47  def test_controllers_converge():
48      ref = RefPath()
49      for c in (Stanley(), AdaptiveMPC(adapt=True)):
50          x, y, th, _ = ref.at(0); veh = Vehicle(x, y + 1.0, th, 10.0); s = 0.0
51          for _ in range(300):
52              s, l, _ = ref.project(veh.x, veh.y, s); xr, yr, thr, k = ref.at(s)
53              r = dict(x=xr, y=yr, th=thr, kappa=k, vN=np.full(15, 10.0), kN=np.array([ref.at(s + 1.0 * q)[3] for q in range(1, 16)]))
54              d = c.command((veh.x, veh.y, veh.th), veh.v, r); yr_ = veh.step(d, 0.0)
55              if hasattr(c, "estimate"): c.estimate(veh.v, yr_, d, 0.05)
56          assert abs(l) < 0.1, c.name
57
58
59  if __name__ == "__main__":
60      for name, fn in sorted(globals().items()):
61          if name.startswith("test_"):
```

```
            fn(); print("PASS", name)
```

Listing 14: tests/test_datasets.py

```
"""Run: python -m tests.test_datasets  (validates the KITTI-format loader and the metrics on synthetic data)."""
import os, sys, tempfile
import numpy as np
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from datasets import load_velodyne, load_poses, scan_to_2d, ate_rmse, rpe_translation, kitti_sequence


def main():
    rng = np.random.default_rng(0)
    with tempfile.TemporaryDirectory() as root:
        os.makedirs(f"{root}/sequences/00/velodyne"); os.makedirs(f"{root}/poses")
        pts = rng.normal(0, 10, (2000, 4)).astype(np.float32); pts[:, 2] = rng.uniform(-2, 2, 2000)
        for i in range(3):
            pts.tofile(f"{root}/sequences/00/velodyne/{i:06d}.bin")
        P = np.array([np.r_[np.eye(3).ravel().reshape(3, 3), [[i * 1.0], [0], [0]]].reshape(-1) if False else
                      np.hstack([np.eye(3), [[i * 1.0], [0.0], [0.0]]]).ravel() for i in range(3)])
        np.savetxt(f"{root}/poses/00.txt", P)
        files, poses = kitti_sequence(root)
        assert len(files) == 3 and poses.shape == (3, 4, 4) and abs(poses[2, 0, 3] - 2.0) < 1e-9
        cloud = load_velodyne(files[0]); assert cloud.shape == (2000, 4)
        assert 0 < len(scan_to_2d(cloud)) <= 120
    gt = np.c_[np.linspace(0, 100, 200), 5 * np.sin(np.linspace(0, 6, 200))]
    th = 0.3; R = np.array([[np.cos(th), -np.sin(th)], [np.sin(th), np.cos(th)]])
    est = gt @ R.T + [3.0, -2.0]
    assert ate_rmse(est, gt, align=True) < 1e-9                    # pure rigid offset is removed by alignment
    assert ate_rmse(est, gt, align=False) > 1.0
    noisy = gt + rng.normal(0, 0.2, gt.shape)
    assert 0.15 < ate_rmse(noisy, gt) < 0.35 and rpe_translation(noisy, gt) > 0
    print("test_datasets: OK")


if __name__ == "__main__":
    main()
```

Listing 15: run_all.py

```
"""
Reproduce everything: tests -> experiments -> figures -> results/*.log (+ results/*.json).
  python run_all.py --quick   tests + localisation + BT batch + closed loop + latency  (~2 min)
  python run_all.py --full    additionally BT grid verification and fault-injection campaigns (~20 min)
Independent experiments run in parallel processes; each log is the exact stdout of its script.
"""
import os, subprocess, sys, time

JOBS_QUICK = [
    ("localization", [sys.executable, "-u", "run_localization.py", "--plot"]),
    ("bt_batch", [sys.executable, "-u", "behavior_tree.py", "--plot"]),
    ("closed_loop", [sys.executable, "-u", "closed_loop.py", "--plot"]),
    ("latency", [sys.executable, "-u", "deploy/embedded_bench.py"]),
]
JOBS_FULL = [
    ("bt_verification", [sys.executable, "-u", "verify_bt.py"]),
    ("fault_injection", [sys.executable, "-u", "fault_injection.py"]),
]


def main():
    os.makedirs("results", exist_ok=True); os.makedirs("figures", exist_ok=True)
    for t in ("tests.test_all", "tests.test_datasets"):
        r = subprocess.run([sys.executable, "-m", t], capture_output=True, text=True)
        open(f"results/{t.split('.')[-1]}.log", "w").write(r.stdout + r.stderr)
        print(("PASS " if r.returncode == 0 else "FAIL ") + t)
        if r.returncode:
            sys.exit(r.stderr)
    jobs = JOBS_QUICK + (JOBS_FULL if "--full" in sys.argv else [])
    procs = []
    for name, cmd in jobs:
        f = open(f"results/{name}.log", "w")
        procs.append((name, subprocess.Popen(cmd, stdout=f, stderr=subprocess.STDOUT), f, time.time()))
    for name, p, f, t0 in procs:
        p.wait(); f.close()
        print(f"{'done' if p.returncode == 0 else 'FAILED'} {name} ({time.time() - t0:.0f}s) -> results/{name}.log")


if __name__ == "__main__":
    main()
```

Listing 16: deploy/Dockerfile

```
# Reproducible CPU image for tests + experiments (multi-arch: works on x86-64 and aarch64 / Jetson-class hosts)
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "run_all.py", "--quick"]
```

C Raw Experiment Logs

Listing 17: results/localization.log

```
HD map: 2133 points, route length 412 m

Scenario: nominal init (1.4 m, 5 deg off)  (5 seeds, mean of per-run metrics)
method                    RMSE pos [m]   p95 [m]   max [m]  RMSE yaw [deg]  converged
Dead reckoning                   35.01     50.39     51.27           32.48         0%
NDT only                          0.11      0.20      1.41            0.26       100%
PF (NDT likelihood)               2.89      4.74      4.87            3.80         0%
NDT-PF (ours)                     0.30      0.55      1.41            0.54       100%

Scenario: hard init (5 m, 20 deg off)  (5 seeds, mean of per-run metrics)
method                    RMSE pos [m]   p95 [m]   max [m]  RMSE yaw [deg]  converged
Dead reckoning                   53.78     79.28     80.22           46.88         0%
NDT only                          0.49      0.21      5.02            1.57       100%
PF (NDT likelihood)               3.04      4.99      5.34            4.25         0%
NDT-PF (ours)                     0.50      0.81      5.00            1.54       100%
saved localization_results.png
```

Listing 18: results/bt_batch.log

```
300 randomised episodes (random lead speed/position, oncoming car, circulating traffic)
  completed route        : 100.0%
  episodes with collision: 0.0%
  overtakes performed    : 0.72 per episode (72% of episodes)
  mean route time        : 57.5 s
  min gap (median / 5th) : 9.2 m / 6.4 m
  episodes that had to wait at yield line: 35%
saved bt_episode.png
```

Listing 19: results/bt_verification.log

```
structure: {'leaves': ['approaching_roundabout', 'before_yield_line', 'conflict_ahead', 'yield', 'enter_roundabout', 'blocked_by_slow_lead', 'overtake_safe', 'go_left', 'pass_lead', '
       return_right', 'cruise'], 'depth': 4, 'root_total': True}

[A overtaking] 3575 scenarios in 228s
  violations: none | incomplete routes: 0

[B roundabout yield] 1856 scenarios in 88s
  violations: none | incomplete routes: 0
```

Listing 20: results/fault_injection.log

```
== (A) localisation faults, 5 seeds ==
baseline             NDT rmse   0.11 max   1.41 conv 100% | NDT-PF rmse   0.30 max   1.41 conv 100% | det recall n/a FA 0.000
blackout_5s          NDT rmse   0.16 max   1.41 conv 100% | NDT-PF rmse   0.35 max   1.41 conv 100% | det recall n/a FA 0.056
outliers_40pct       NDT rmse   0.14 max   1.41 conv 100% | NDT-PF rmse   0.32 max   1.41 conv 100% | det recall n/a FA 0.000
gyro_bias_0.03       NDT rmse   0.12 max   1.41 conv 100% | NDT-PF rmse   0.21 max   1.41 conv 100% | det recall n/a FA 0.000
sensor_noise_0.3m    NDT rmse   0.16 max   1.41 conv 100% | NDT-PF rmse   0.29 max   1.41 conv 100% | det recall n/a FA 0.000
map_change_33pct     NDT rmse   0.14 max   1.41 conv 100% | NDT-PF rmse   0.41 max   1.41 conv 100% | det recall n/a FA 0.000
blackout_20s         NDT rmse  29.64 max  64.60 conv  20% | NDT-PF rmse   2.77 max  10.76 conv 100% | det recall 0.70 FA 0.114
outliers_70pct       NDT rmse   0.23 max   1.65 conv 100% | NDT-PF rmse   0.38 max   1.46 conv 100% | det recall n/a FA 0.941
map_change_67pct     NDT rmse   0.17 max   1.41 conv 100% | NDT-PF rmse   0.35 max   1.41 conv 100% | det recall n/a FA 0.000
severe_combo         NDT rmse  61.09 max 107.05 conv   0% | NDT-PF rmse  54.47 max  92.01 conv   0% | det recall 1.00 FA 0.191

== (B) decision-making faults, 200 episodes each ==
none                     collisions   0.0% ->   0.0% (with supervisor) | completion 100.0% / 100.0%
latency_0.5s             collisions   0.0% ->   0.0% (with supervisor) | completion 100.0% / 100.0%
latency_1.0s             collisions   0.0% ->   0.0% (with supervisor) | completion 100.0% / 100.0%
oncoming_dropout_3s      collisions  86.5% ->   0.0% (with supervisor) | completion 100.0% / 100.0%
oncoming_speed_x0.6      collisions   0.0% ->   0.0% (with supervisor) | completion 100.0% / 100.0%
cv_timing_sigma_1.5s     collisions   2.0% ->   2.0% (with supervisor) | completion 100.0% / 100.0%
```

Listing 21: results/closed_loop.log

```
condition               controller           done  RMS ey  max ey  coll  clr[m]  plan ms
nominal                 Pure pursuit         True   0.112   0.51     0    1.44     27.1
nominal                 Stanley              True   0.070   0.26     0    1.29     26.4
nominal                 MPC (fixed model)    True   0.060   0.24     0    1.18     26.2
nominal                 Adaptive MPC         True   0.060   0.24     0    1.17     27.4
gain_0.7+latency_0.1s   Pure pursuit         True   0.160   0.61     0    1.39     29.6
gain_0.7+latency_0.1s   Stanley              True   0.117   0.51     0    1.31     27.0
gain_0.7+latency_0.1s   MPC (fixed model)    True   0.090   0.36     0    1.24     27.8
gain_0.7+latency_0.1s   Adaptive MPC         True   0.897   3.08     0    0.81     27.1
loc_noise_0.3m          Pure pursuit         True   0.193   0.64     0    1.04     27.9
loc_noise_0.3m          Stanley              True   0.155   0.62     0    0.79     26.3
loc_noise_0.3m          MPC (fixed model)    True   0.145   0.62     0    1.35     26.5
loc_noise_0.3m          Adaptive MPC         True   0.145   0.62     0    1.35     29.7
all_faults              Pure pursuit         True   0.254   0.91     0    1.06     29.4
all_faults              Stanley              True   0.205   0.67     0    1.49     26.8
all_faults              MPC (fixed model)    True   0.217   0.59     0    1.60     25.7
all_faults              Adaptive MPC        False   0.618   4.57     0    0.40     13.0
saved figures/closed_loop.png
```

Listing 22: results/latency.log

```
pf_update_100_particles      p50     1.06 ms   p95     1.20 ms
pf_update_300_particles      p50     3.64 ms   p95     3.93 ms
pf_update_1000_particles     p50    11.95 ms   p95    13.86 ms
ndt_align_6_iters            p50     1.92 ms   p95     2.27 ms
ndt_pf_full_cycle_300        p50     5.61 ms   p95     6.22 ms
lattice_plan                 p50    20.48 ms   p95    30.12 ms
qp_trajectory_opt            p50     0.88 ms   p95     1.00 ms
mpc_step                     p50     0.18 ms   p95     0.24 ms
bt_tick                      p50     0.03 ms   p95     0.04 ms
budget localisation_10Hz     p95    6.22 / 100 ms  OK
budget planning_2Hz          p95   30.12 / 500 ms  OK
budget control_20Hz          p95    0.24 / 50 ms  OK
budget decision_10Hz         p95    0.04 / 100 ms  OK
```

Listing 23: results/test_all.log

```
PASS test_bt_never_overtakes_into_oncoming
PASS test_bt_structure_and_safety
PASS test_controllers_converge
PASS test_emergency_when_boxed_in
PASS test_ndt_recovers_offset
PASS test_planner_collision_free_and_feasible
```